WASTE MANAGEMENT SYSTEM

PROJECT REPORT

Submitted in partial fulfilment of the requirements for the completion of

PBCST304: OBJECT ORIENTED PROGRAMMING

Submitted by

Aman Fayad | KTU ID: TVE25CS023 | Roll: 12
Joseph Fidel K M | KTU ID: TVE25CS068 | Roll: 31
Ayshwaria Gopakumar | KTU ID: TVE25CS041 | Roll: 21
Nivedya Sunil | KTU ID: TVE25CS101 | Roll: 51
Nida Rahman C | KTU ID: TVE25CS097 | Roll: 47



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
COLLEGE OF ENGINEERING TRIVANDRUM

October 2026

DECLARATION

We, the undersigned, declare that the project report entitled **Waste Management System**, submitted in fulfilment of the requirements for PBCST304: Object Oriented Programming, describes the work carried out by our team at the College of Engineering Trivandrum. The report records the design, implementation, and verification of the updated Java Swing desktop application connected to MySQL using JDBC. Member details below are transcribed from the supplied presentation PDF.

Student and KTU ID	Roll	Signature
Aman Fayad (TVE25CS023)	12	
Joseph Fidel K M (TVE25CS068)	31	
Ayshwaria Gopakumar (TVE25CS041)	21	
Nivedya Sunil (TVE25CS101)	51	
Nida Rahman C (TVE25CS097)	47	

The team should check the printed identifiers and sign this declaration before submission.

ACKNOWLEDGEMENT

We express our sincere thanks to the Department of Computer Science and Engineering, College of Engineering Trivandrum, for providing the academic setting in which this project was completed. We are grateful to the faculty members who introduced the object-oriented programming concepts applied in this work. We also acknowledge Clean CET for the campus-cleanliness context that informed the requirements. Finally, we thank our teammates for their collaboration in planning, implementation, testing, and presentation preparation.

ABSTRACT

The Waste Management System is a desktop application developed as an Object Oriented Programming mini-project for PBCST304. It provides a graphical interface using Java Swing and stores collection records in MySQL through JDBC and MySQL Connector/J. The model is organized around an abstract Waste class and three concrete categories: organic, recyclable, and hazardous waste. The application supports authenticated member and administrator roles, timestamped record entry, admin-approved completion requests, and date-filtered reports by category, collector, and actual collection area.

Members can add records, update their own pending entries, request admin approval to mark a collection complete, flag their own pending records as invalid, and view their individual contribution. The administrator can view all records, approve or reject requests, mark records collected directly, delete records, and access overall, member-wise, and area-wise reports. Approval updates request and waste status in one transaction; only completed records count in reports. Collection outside a member's assigned territory is classified as bonus in the member report, while the area report follows actual location and sums recorded kilograms. Totals are derived from the canonical waste table rather than duplicated summaries. Local MySQL installations do not synchronize automatically, and kilograms are not a scientifically measured pollution index.

TABLE OF CONTENTS

DECLARATION i

ACKNOWLEDGEMENT ii

ABSTRACT iii

TABLE OF CONTENTS iv

LIST OF FIGURES vi

1 Introduction 1

1.1 Project Background . . . 1

1.2 Project Overview . . . 1

1.3 Scope . . . 1

2 Problem Definition 1

2.1 Problem Statement . . . 1

2.2 Objectives . . . 2

2.3 User Roles . . . 2

3 Methodology 3

3.1 Layered Design . . . 3

3.2 Application Workflow . . . 3

3.3 Object-Oriented Design . . . 3

3.3.1 Abstraction and inheritance . . . 3

3.3.2 Encapsulation and polymorphism . . . 3

3.3.3 Other Java concepts . . . 3

3.4 Record State and Permissions . . . 4

3.5 Database Design . . . 4

3.6 Report Calculation . . . 4

4 Implementation Details 4

4.1 System Requirements . . . 4

4.2 Experimental Setup . . . 4

4.3 Packages and Classes . . . 5

4.4 Database Fields . . . 6

4.5 Authentication and Configuration . . . 6

4.6 Build and Run . . . 7

4.7 Security and Operational Considerations . . . 7

5 Results **7**

5.1 Verification Performed . . . 7

5.2 Functional Test Cases . . . 8

5.3 Observed Results . . . 8

6 Conclusion and Future Works **9**

6.1 Conclusion . . . 9

6.2 Limitations . . . 9

6.3 Future Enhancements . . . 9

References **9**

Appendix A: Database Setup and Inspection **9**

Appendix B: Source Code **10**

B.1 Main . . . 10

B.2 Waste model . . . 11

B.3 OrganicWaste . . . 12

B.4 RecyclableWaste . . . 12

B.5 HazardousWaste . . . 12

B.6 AppUser . . . 13

B.7 RegistrationResult . . . 13

B.8 ReportSummary . . . 13

B.9 ReportRow . . . 14

B.10 DatabaseManager . . . 15

B.11 PasswordHasher . . . 18

B.12 AuthenticationManager . . . 19

B.13 UserManager . . . 20

B.14 WasteManager . . . 23

B.15 ReportManager . . . 27

B.16 AnnouncementManager . . . 29

B.17 Announcement model . . . 30

B.18 LoginFrame . . . 31

B.19 CleanButtonUI . . . 33

B.20 WasteManagementGUI . . . 34

LIST OF FIGURES

Figure	Title	Page
1	High-level architecture of the Waste Management System	1

1 Introduction

1.1 Project Background

Campus waste collection involves recording what was collected, its category, quantity, actual collection location, responsible member, and collection time. Scattered notes make records harder to find, review, and summarize. This project addresses that bounded record-management problem with a desktop application and relational database.

The system was developed in the Clean CET campus-cleanliness context at the College of Engineering Trivandrum. It is a record system: it does not use sensors to detect waste, dispatch collection vehicles, optimize routes, or calculate pollution from air, soil, or water measurements.

1.2 Project Overview

Layer	Responsibility	Implementation
Presentation	Accept input and show records/reports	Java Swing frames, panels, tables, event listeners
Model and rules	Represent users, waste, and report data	Java model classes and manager classes
Data access	Authenticate, query, and update records	JDBC, PreparedStatement, ResultSet
Persistence	Keep data across application runs	MySQL database and Connector/J driver

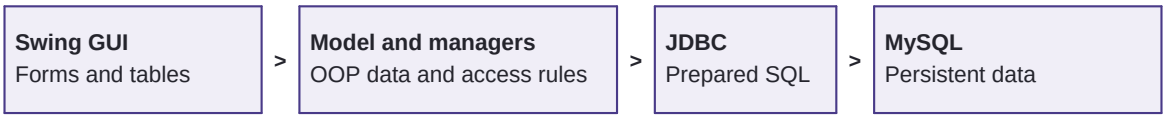


Figure 1. High-level architecture of the Waste Management System

Startup initializes the database and opens LoginFrame. After authentication, WasteManagementGUI presents role-scoped records and reports. A GUI action calls a manager, which runs SQL through JDBC and refreshes the view.

1.3 Scope

The current desktop application includes login, role-aware add/view/update/delete actions, search, timestamps, pending/completed/invalid states, member completion requests and an admin approval queue, date-filtered reports, member and area summaries, category totals, a Swing chart, CSV export based on the applied date range, logout, and an administrator broadcast inbox with per-member unread/read status.

2 Problem Definition

2.1 Problem Statement

The project provides a consistent way to maintain campus waste-collection records across assigned locations. Each record needs category, description, kilograms, actual location, credited collector, and collection timestamp. The system separates entry from administrative review so incomplete or incorrect records do not count as confirmed collection. Members should see only their own contribution; the administrator needs the wider operational and reporting view.

2.2 Objectives

No.	Objective
1	Build a native desktop GUI with Java Swing.
2	Persist collection records in MySQL and connect through JDBC.
3	Demonstrate abstraction, inheritance, encapsulation, and polymorphism in the waste model.
4	Authenticate administrator and member accounts with role-aware access.
5	Record waste category, quantity, actual area, collector, and collection timestamp.
6	Separate pending, completed, and invalid records; route member completion requests for admin review.
7	Provide personal and administrator-only overall/member/area reports.
8	Calculate reports from source records so totals refresh consistently.
9	Allow the administrator to broadcast updates and members to track unread notices.

2.3 User Roles

The seeded account list contains one administrator and thirteen members. The names and territories below reflect the current account configuration.

Member	Assigned location
Aman Fayad	Civil canteen
Joseph Fidel	Mech front
Nivedya	Dhwani
Ayshwaria	Gazebo
Nida	CS front
Arya	Golden walkway
Ahban	Electrical gate
Sidharth	Indoor court
Heera	Archie corner
Keerthana	College front
Farzeen	DJ hall
Merin	CGPU hall
Shyam	CETAA hall

Action	Member	Administrator
View records	Own authored/credited records	All records
Add and update	Add as self; update own pending records	Add/assign and update pending records
Mark invalid	Own pending records	Any pending record

Action	Member	Administrator
Request completion / approve	Request own pending record	Approve/reject; can also mark collected directly
Delete	Not permitted	Permitted, with confirmation
Reports	Own contribution only	Overall, each member, and each area
Announcements	Read notices and mark them read	Publish notices to all members

3 Methodology

3.1 Layered Design

Model classes hold domain data and do not manage screen layout or SQL. Swing GUI classes own user interaction and presentation. DAO/manager classes own database operations, authentication, report queries, and role/ownership rules. DatabaseManager centralizes connection settings, startup table creation/migration, and account seeding. This division is small enough for a course project while keeping responsibilities understandable.

3.2 Application Workflow

The Java entry point shows a startup status window and initializes the database in SwingWorker.doInBackground(). On success, its done() method opens LoginFrame on Swing's Event Dispatch Thread. LoginFrame passes credentials to AuthenticationManager, which verifies a salted PBKDF2 hash. On success, an AppUser object is passed to WasteManagementGUI. GUI events validate forms, invoke the appropriate manager, and refresh the view. A member completion request appears in the admin approval queue; WasteManager approves the request and completes the waste record in one transaction. The queue count refreshes every 15 seconds. ReportManager calculates aggregates; AnnouncementManager handles admin broadcasts and member read receipts. Logout disposes the authenticated dashboard and displays a new login window.

3.3 Object-Oriented Design

3.3.1 Abstraction and inheritance

Waste is an abstract superclass with shared record fields and the abstract method getWasteCategory(). OrganicWaste, RecyclableWaste, and HazardousWaste extend Waste and provide category-specific implementations.

3.3.2 Encapsulation and polymorphism

Model fields are private and are accessed through getters and setters. That is the project's direct example of encapsulation. The category method can be called through a Waste reference but returns a value based on the runtime subclass, demonstrating polymorphism. Shared data and behavior live in the parent instead of being repeated.

3.3.3 Other Java concepts

Classes are instantiated for users, waste entries, and report rows. Packages separate model, dao, gui, and main responsibilities. SQL and parsing errors are caught and reported. Swing ActionListeners connect button events to GUI methods. PreparedStatement binds user data to parameterized SQL. Swing components are created on the Event Dispatch Thread; lengthy operations on that thread can make a window unresponsive, so background database work is a production improvement.

3.4 Record State and Permissions

New records use PENDING. A member or administrator can flag an eligible pending record INVALID; the record is retained and excluded from completed reports. Members may request completion for their own pending records. The administrator may approve/reject these requests or mark a selected pending record collected directly. Approval changes request and waste status in one transaction; only completed entries count in reports. Editing or invalidating a record cancels an outstanding request. The administrator can delete records. Member ownership rules are enforced in the manager layer as well as by hiding unavailable controls in the interface.

created_by records which account entered a row; collected_by identifies the member credited for the collection. This allows an administrator to enter a record for a member. The report compares actual location to the member's assigned area: off-territory collection is bonus in the member view, while the area report attributes that same collection to its actual location.

3.5 Database Design

The schema uses app_users for accounts, waste for collection records, announcements for administrator broadcasts, announcement_reads for per-user read receipts, and completion_requests for member-submitted completion requests and admin review outcomes. The user table contains username, display name, role, assigned area, salt, password hash, and active state. Waste stores category, description, quantity, actual location, status, author, collector, collection timestamp, optional invalid reason, and database-managed creation/update times. Startup applies additive migrations and creates missing communication tables without dropping existing waste contents.

Reports are not stored as one table per member. ReportManager uses SQL aggregation (SUM, COUNT, grouping, joins, and date bounds) over canonical waste rows. It avoids duplicate totals that could become stale after updates or status changes. PreparedStatement binds values, ResultSet reads query results, and Connector/J provides the JDBC driver implementation for MySQL [1, 2].

3.6 Report Calculation

For a selected date interval, report queries use only completed records whose collection timestamp lies in the interval. Apply Dates refreshes the summary and member/area tables; CSV export uses the last successfully applied range. Category, collector, and actual location determine grouping. Area totals sum kilograms and count records at the actual location. They describe recorded waste accumulation, not area population or a scientifically measured pollution score. Assigned-versus-bonus classification is a member-report view and does not add another physical collection to the overall total.

4 Implementation Details

4.1 System Requirements

The application requires a Java runtime and a running MySQL Server. Rebuilding source requires a JDK. The MySQL Connector/J JAR is bundled under DesktopApplication/lib. The GUI uses Swing and AWT; no web framework or JavaFX is used by the desktop program. A database account needs privileges for initial database/table creation and migration.

4.2 Experimental Setup

The verified development environment was macOS 26.6.2 (Apple silicon), Eclipse Temurin OpenJDK 21.0.5 LTS, and the local Homebrew MySQL server reported as version 26.7.0. The project was compiled with the supplied shell build script and exercised against the local database. The source includes Windows batch launch/build instructions, but execution on a physical Windows computer was not independently verified in this audit.

4.3 Packages and Classes

Package or class	Responsibility
model.Waste	Abstract parent with shared private fields and category contract.
OrganicWaste, RecyclableWaste, HazardousWaste	Concrete categories overriding getWasteCategory().
AppUser	Authenticated identity, role, assigned area, DOB, register number, and department.
RegistrationResult	New member identity and one-time display value for the generated password.
ReportSummary, ReportRow	Typed report totals and member/area output rows.
DatabaseManager	JDBC configuration, schema creation/migration, account seeding.
PasswordHasher, AuthenticationManager	Salted PBKDF2 password hashing and credential verification.
UserManager	Registration validation, random area allocation, admin-only member search, and area management.
WasteManager	CRUD, role/ownership checks, and transactional completion-request approval.
ReportManager	Date-filtered aggregate report queries.
AnnouncementManager	Admin-only broadcast publishing, inbox queries, unread counts, and read receipts.
Announcement	Immutable announcement data shown in the inbox.
LoginFrame	Swing login and registration form with sign-in event handling.
CleanButtonUI	Shared button painter for consistent fills and readable interaction states.
WasteManagementGUI	Dashboard, records, admin approval queue, applied-date reports, CSV export, announcements, and logout.
Main	Application entry point and startup error handling.

4.4 Database Fields

Table	Fields	Meaning
app_users	username, display_name, role	Login identity and access role.
app_users	assigned_area	Member's regular territory.
app_users	date_of_birth, register_number, department	Registration profile; register number is unique.
app_users	password_salt, password_hash, active	Salted password hash and account state.
waste	id, waste_type, description, quantity	Record identity, category, notes, and kg.
waste	location, status, collected_at	Actual place, review state, collection date/time.
waste	created_by, collected_by	Account that entered the row and member credited.
waste	invalid_reason, created_at, updated_at	Optional invalid explanation and audit timestamps.
announcements	title, message, created_by, created_at	Administrator-to-member broadcasts.
announcement_reads	announcement_id, username, read_at	Per-member read receipts and unread counts.
registration_areas	area_name, assigned_username	Available areas for random member allocation; admin can add areas.
completion_requests	waste_id, requested_by, status, reviewed_by, timestamps	Member requests and admin review history.

4.5 Authentication and Configuration

The default JDBC host is localhost:3306 and the database is waste_management_db. WASTE_DB_SERVER, WASTE_DB_NAME, WASTE_DB_USER, and WASTE_DB_PASSWORD can override connection configuration. On a fresh database, WASTE_ADMIN_INITIAL_PASSWORD is read once to initialize the admin hash; it should be set privately and never committed. Password hashes are stored with per-account salts, not plaintext passwords. Join Clean CET collects full name, date of birth, CET register number, and department; the lowercase first name becomes the username. Registration checks the register-number format and uniqueness, but does not verify it with an official registry. The requested username_clean password is predictable, shown once, and unsuitable for production. DOB and register numbers are personal data and require informed consent.

The fifteen initial registration locations are Industrial department front, EC department front, Rohith square, Sargam stage, Library front, Ohm's villa, Industrial base, College ground, Research block, Old EC department, Hydraulics lab front, Back gate, Mh front, Lh front, and Flag post. Each location is allocated to at most one newly registered member until an administrator adds more locations.

Localhost means each laptop has a separate database by default. Shared team data requires all clients to use a common reachable MySQL host with secure networking and appropriate privileges. The Java desktop app is not a hosted synchronization service.

4.6 Build and Run

On Windows, run build.bat and then run.bat from DesktopApplication. On macOS/Linux, run ./build.sh followed by ./run.sh. The build compiles against the bundled Connector/J JAR and creates dist/WasteManagementSystem.jar. A running MySQL service is required when launching the application.

4.7 Security and Operational Considerations

PBKDF2-derived salted hashes protect stored passwords. Join Clean CET collects full name, date of birth, CET register number, and department; the lowercase first name becomes the username. Registration validates the register-number format and uniqueness but does not check an official registry. The requested username_clean password is predictable, displayed once, and suitable only for a classroom prototype. DOB and register numbers are personal data and should be collected only with informed consent. Role checks in the desktop client and DAO layer do not protect someone with direct database credentials. A production system would put authorization behind a trusted service, protect secrets, implement account recovery, and maintain audit logs and backups.

5 Results

5.1 Verification Performed

The Java source compiled successfully with the supplied build script. A JDBC connection using environment-configured local MySQL credentials was verified; secrets are not included in this report. MySQL integration checks confirmed the original thirteen USER accounts and one ADMIN remain intact. A temporary registration test verified lowercase first-name account creation, DOB/register/department persistence, assignment of an available area, successful authentication, duplicate-register rejection, and admin-only profile search; the test account was deleted afterward. Existing users' new profile fields remain blank and display as not provided. Additional MySQL checks covered authorization, update, admin-only completion/deletion, invalid states, reports, date filtering, bonus attribution, completion approval/rejection, and direct admin completion.

5.2 Functional Test Cases

Test	Input/action	Expected result
Login	Valid member credentials	Member identity and assigned area load.
Bad password	Incorrect password	Access denied; no workspace opens.
Member restrictions	Attempt complete or delete	Operation is denied; admin-only controls are unavailable.
Add record	Category, positive kg, location, timestamp	Pending record is stored and shown.
Invalid record	Member flags own pending entry	Status is INVALID; excluded from completed reports.
Request completion	Member requests completion	Waste stays pending and appears in admin approval queue.
Approval	Admin approves member request	Request and waste statuses update atomically; reports include it.
Reject and retry	Admin rejects; member requests again	Waste stays pending and resubmission is accepted.
Direct admin collection	Admin selects pending row and marks collected	Status changes to COMPLETED; reports include it.
Bonus attribution	Member records collection outside assigned area	Member report calls it bonus; area report uses actual location.
Date filter	Apply start/end dates	Dashboard aggregates completed records in the applied range.
Logout	Log out and sign in as another role	Login returns; next dashboard uses the newly authenticated role.
Announcements	Admin publishes; member opens notice	Member sees unread notice, read receipt updates, member cannot publish.
CSV export	Apply a date range, then export	Export uses the last successfully applied range.
Join Clean CET	Submit a valid new member profile	Lowercase first-name username, salted password hash, random unused area, and sign-in-ready account are created.
Duplicate registration	Reuse an existing register number or first-name username	Registration is rejected without assigning or consuming an area.
Member directory permissions	Open directory as admin, then attempt member access	Admin sees member profile/area/status; regular member search is denied.

5.3 Observed Results

The administrator interface shows total and category summary values, member-wise contribution, and area-wise collection with a Swing chart and CSV export. The administrator can review requests in a queue whose count refreshes every 15 seconds, or mark records collected directly. Admin broadcasts and member read receipts remain available. Members see their own report and progress. Reports include approved completed records only and show recorded kilograms by actual location; these are not measurements of pollution or population.

6 Conclusion and Future Works

6.1 Conclusion

The project delivers a role-based campus waste register as a Java Swing desktop application with MySQL persistence through JDBC. Its structure makes the required OOP concepts visible, separates UI/model/data responsibilities, and supports authentication, timestamped entries, review states, and report generation. SQL aggregation over one source-of-truth waste table keeps member, area, and overall summaries aligned with current records.

6.2 Limitations

Default installations are local and do not synchronize; team-wide requests require a shared reachable MySQL server. Announcements are administrator-to-member broadcasts, not two-way chat, and the approval count polls every 15 seconds rather than using push. Reports summarize kilograms by category and location, not a validated pollution score or population estimate. Initial member credentials follow a predictable convention. Client-side authorization can be bypassed by direct database access. Synchronous JDBC work can block the Swing event thread on a slow server. No formal performance, usability, or environmental-impact study is claimed.

6.3 Future Enhancements

Future work could add a secured shared service, push notifications, first-login password changes and account recovery, audit logs, backups, and background database operations using SwingWorker. Verified environmental measurements could support a carefully defined pollution indicator. PDF export, configurable time zones, and formal usability and performance testing are also possible extensions.

References

[1] Oracle Corporation and MySQL, *MySQL Connector/J Developer Guide: Overview*. https://dev.mysql.com/doc/connector-j/en/connector-j-overview.html (accessed 17 October 2026).

[2] Oracle Corporation, *Using Prepared Statements*, The Java Tutorials: JDBC Database Access. https://docs.oracle.com/javase/tutorial/jdbc/basics/prepared.html (accessed 17 October 2026).

[3] Oracle Corporation, *The Event Dispatch Thread*, The Java Tutorials: Creating a GUI With Swing. https://docs.oracle.com/javase/tutorial/uiswing/concurrency/dispatch.html (accessed 17 October 2026).

[4] Oracle Corporation, *Abstract Methods and Classes*, The Java Tutorials: Inheritance. https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html (accessed 17 October 2026).

Appendix A: Database Setup and Inspection

The application creates or migrates its tables at startup. The following read-oriented SQL commands can be used to inspect the resulting schema and data.

```
USE waste_management_db;
SHOW TABLES;
DESCRIBE app_users;
DESCRIBE waste;
DESCRIBE announcements;
DESCRIBE announcement_reads;
DESCRIBE completion_requests;
SELECT username, display_name, role, assigned_area FROM app_users;
SELECT id, waste_type, quantity, location, status, collected_at, collected_by
FROM waste ORDER BY collected_at DESC;
SELECT waste_type, SUM(quantity) AS kilograms
FROM waste WHERE status = 'COMPLETED' GROUP BY waste_type;
```

Appendix B: Source Code

The source files below are read directly from the updated DesktopApplication/src tree. This appendix therefore corresponds to the desktop application version described in this report.

B.1 Main

```
package main;

import dao.DatabaseManager;
import gui.LoginFrame;

import javax.swing.JOptionPane;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.SwingWorker;
import javax.swing.SwingUtilities;
import java.awt.BorderLayout;
import java.awt.Dimension;
import java.util.concurrent.ExecutionException;
import java.sql.SQLException;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame startup = new JFrame("Waste Management System");
            JLabel status = new JLabel("Connecting to MySQL and preparing the workspace...",
      JLabel.CENTER);
            startup.add(status, BorderLayout.CENTER);
            startup.setSize(new Dimension(440, 100));
            startup.setLocationRelativeTo(null);
            startup.setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
            startup.setVisible(true);

            SwingWorker<Void, Void> initializer = new SwingWorker<Void, Void>() {
                @Override
                protected Void doInBackground() throws SQLException {
                    DatabaseManager.initializeDatabase();
                    return null;
                }

                @Override
                protected void done() {
                    startup.dispose();
                    try {
                        get();
                        new LoginFrame().setVisible(true);
                    } catch (InterruptedException exception) {
                        Thread.currentThread().interrupt();
                        showDatabaseError(exception);
                    } catch (ExecutionException exception) {
                        showDatabaseError(exception.getCause());
                    }
                }
            };
            initializer.execute();
        });
    }

    private static void showDatabaseError(Throwable exception) {
        JOptionPane.showMessageDialog(
                null,
                "Could not connect to MySQL.\n\n"
                        + "Please make sure MySQL is installed and running.\n"
                        + "Check the WASTE_DB_USER and WASTE_DB_PASSWORD settings.\n\n"
                        + "Error: " + exception.getMessage(),
                "Database Connection Error",
                JOptionPane.ERROR_MESSAGE
        );
    }
}
```

B.2 Waste model

```
package model;

// Abstraction: Waste is abstract because a general "Waste" record needs a real category.
public abstract class Waste {
    // Encapsulation: fields are private and accessed through getters/setters.
    private int id;
    private String description;
    private double quantity;
    private String location;
    private String status;
    private String collectedBy;
    private String createdBy;
    private String collectedAt;
    private String invalidReason;
    private boolean completionRequested;
    private String completionRequestStatus;

    public Waste() {
    }

    public Waste(int id, String description, double quantity, String location, String status) {
        this.id = id;
        this.description = description;
        this.quantity = quantity;
        this.location = location;
        this.status = status;
    }

    // Polymorphism: each child class gives its own category.
    public abstract String getWasteCategory();

    public String getCollectedBy() { return collectedBy; }
    public void setCollectedBy(String collectedBy) { this.collectedBy = collectedBy; }
    public String getCreatedBy() { return createdBy; }
    public void setCreatedBy(String createdBy) { this.createdBy = createdBy; }
    public String getCollectedAt() { return collectedAt; }
    public void setCollectedAt(String collectedAt) { this.collectedAt = collectedAt; }
    public String getInvalidReason() { return invalidReason; }
    public void setInvalidReason(String invalidReason) { this.invalidReason = invalidReason; }
    public boolean isCompletionRequested() { return completionRequested; }
    public void setCompletionRequested(boolean completionRequested) { this.completionRequested =
      completionRequested; }
    public String getCompletionRequestStatus() { return completionRequestStatus; }
    public void setCompletionRequestStatus(String completionRequestStatus) {
      this.completionRequestStatus = completionRequestStatus; }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public double getQuantity() {
        return quantity;
    }

    public void setQuantity(double quantity) {
        this.quantity = quantity;
    }

    public String getLocation() {
        return location;
    }

    public void setLocation(String location) {
        this.location = location;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}
```

B.3 OrganicWaste

```java
package model;

// Inheritance: OrganicWaste extends the abstract Waste class.
public class OrganicWaste extends Waste {
    public OrganicWaste() {
    }

    public OrganicWaste(int id, String description, double quantity, String location, String status) {
        super(id, description, quantity, location, status);
    }

    @Override
    public String getWasteCategory() {
        return "Organic";
    }
}
```

B.4 RecyclableWaste

```java
package model;

// Inheritance: RecyclableWaste extends the abstract Waste class.
public class RecyclableWaste extends Waste {
    public RecyclableWaste() {
    }

    public RecyclableWaste(int id, String description, double quantity, String location, String status)
      {
        super(id, description, quantity, location, status);
    }

    @Override
    public String getWasteCategory() {
        return "Recyclable";
    }
}
```

B.5 HazardousWaste

```java
package model;

// Inheritance: HazardousWaste extends the abstract Waste class.
public class HazardousWaste extends Waste {
    public HazardousWaste() {
    }

    public HazardousWaste(int id, String description, double quantity, String location, String status) {
        super(id, description, quantity, location, status);
    }

    @Override
    public String getWasteCategory() {
        return "Hazardous";
    }
}
```

B.6 AppUser

```
package model;

public class AppUser {
    private final String username;
    private final String displayName;
    private final String assignedArea;
    private final String role;
    private final String dateOfBirth;
    private final String registerNumber;
    private final String department;

    public AppUser(String username, String displayName, String assignedArea, String role) {
        this(username, displayName, assignedArea, role, null, null, null);
    }

    public AppUser(String username, String displayName, String assignedArea, String role,
                   String dateOfBirth, String registerNumber, String department) {
        this.username = username;
        this.displayName = displayName;
        this.assignedArea = assignedArea;
        this.role = role;
        this.dateOfBirth = dateOfBirth;
        this.registerNumber = registerNumber;
        this.department = department;
    }

    public String getUsername() { return username; }
    public String getDisplayName() { return displayName; }
    public String getAssignedArea() { return assignedArea; }
    public String getRole() { return role; }
    public String getDateOfBirth() { return dateOfBirth; }
    public String getRegisterNumber() { return registerNumber; }
    public String getDepartment() { return department; }
    public boolean isAdmin() { return "ADMIN".equals(role); }
}
```

B.7 RegistrationResult

```
package model;

public class RegistrationResult {
    private final AppUser user;
    private final String temporaryPassword;

    public RegistrationResult(AppUser user, String temporaryPassword) {
        this.user = user;
        this.temporaryPassword = temporaryPassword;
    }

    public AppUser getUser() { return user; }
    public String getTemporaryPassword() { return temporaryPassword; }
}
```

B.8 ReportSummary

```
package model;

public class ReportSummary {
    private final double totalKg;
    private final double organicKg;
    private final double recyclableKg;
    private final double hazardousKg;
    private final int completedRecords;

    public ReportSummary(double totalKg, double organicKg, double recyclableKg,
                         double hazardousKg, int completedRecords) {
        this.totalKg = totalKg;
        this.organicKg = organicKg;
        this.recyclableKg = recyclableKg;
        this.hazardousKg = hazardousKg;
        this.completedRecords = completedRecords;
    }

    public double getTotalKg() { return totalKg; }
    public double getOrganicKg() { return organicKg; }
    public double getRecyclableKg() { return recyclableKg; }
    public double getHazardousKg() { return hazardousKg; }
    public int getCompletedRecords() { return completedRecords; }
}
```

B.9 ReportRow

```
package model;

public class ReportRow {
    private final String label;
    private final double organic;
    private final double recyclable;
    private final double hazardous;
    private final int records;
    private final double assignedKg;
    private final double bonusKg;

    public ReportRow(String label, double organic, double recyclable, double hazardous,
                     int records, double assignedKg, double bonusKg) {
        this.label = label;
        this.organic = organic;
        this.recyclable = recyclable;
        this.hazardous = hazardous;
        this.records = records;
        this.assignedKg = assignedKg;
        this.bonusKg = bonusKg;
    }

    public String getLabel() { return label; }
    public double getOrganic() { return organic; }
    public double getRecyclable() { return recyclable; }
    public double getHazardous() { return hazardous; }
    public int getRecords() { return records; }
    public double getAssignedKg() { return assignedKg; }
    public double getBonusKg() { return bonusKg; }
    public double getTotalKg() { return organic + recyclable + hazardous; }
}
```

B.10 DatabaseManager

```
package dao;

import java.sql.Connection;
import java.sql.DatabaseMetaData;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class DatabaseManager {
    private static final String SERVER_URL = setting("WASTE_DB_SERVER", "jdbc:mysql://localhost:3306/");
    private static final String DATABASE_NAME = setting("WASTE_DB_NAME", "waste_management_db");
    private static final String DATABASE_URL = SERVER_URL + DATABASE_NAME;
    private static final String USERNAME = setting("WASTE_DB_USER", "root");
    private static final String PASSWORD = setting("WASTE_DB_PASSWORD", "");

    private static final String[][] MEMBERS = {
            {"aman", "Aman Fayad", "Civil canteen"},
            {"joseph", "Joseph Fidel", "Mech front"},
            {"nivedya", "Nivedya", "Dhwani"},
            {"ayshwaria", "Ayshwaria", "Gazebo"},
            {"nida", "Nida", "CS front"},
            {"arya", "Arya", "Golden walkway"},
            {"ahban", "Ahban", "Electrical gate"},
            {"sidharth", "Sidharth", "Indoor court"},
            {"heera", "Heera", "Archie corner"},
            {"keerthana", "Keerthana", "College front"},
            {"farzeen", "Farzeen", "DJ hall"},
            {"merin", "Merin", "CGPU hall"},
            {"shyam", "Shyam", "CETAA hall"}
    };
    private static final String[] REGISTRATION_AREAS = {
            "Industrial department front", "EC department front", "Rohith square", "Sargam stage",
            "Library front", "Ohm's villa", "Industrial base", "College ground", "Research block",
            "Old EC department", "Hydraulics lab front", "Back gate", "Mh front", "Lh front", "Flag
      post"
    };

    private static String setting(String name, String fallback) {
        String value = System.getProperty(name);
        if (value == null || value.trim().isEmpty()) value = System.getenv(name);
        return value == null || value.trim().isEmpty() ? fallback : value.trim();
    }

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(DATABASE_URL, USERNAME, PASSWORD);
    }

    public static void initializeDatabase() throws SQLException {
        try (Connection connection = DriverManager.getConnection(SERVER_URL, USERNAME, PASSWORD);
             Statement statement = connection.createStatement()) {
            statement.executeUpdate("CREATE DATABASE IF NOT EXISTS " + DATABASE_NAME
                    + " CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci");
        }

        createTables();
        migrateWasteTable();
        migrateUserTable();
        seedAccounts();
        seedRegistrationAreas();
    }

    private static void createTables() throws SQLException {
        String usersSql = "CREATE TABLE IF NOT EXISTS app_users ("
                + "username VARCHAR(40) PRIMARY KEY, display_name VARCHAR(100) NOT NULL, "
                + "assigned_area VARCHAR(100), role VARCHAR(10) NOT NULL, "
                + "password_salt CHAR(32) NOT NULL, password_hash CHAR(64) NOT NULL, "
                + "date_of_birth DATE NULL, register_number VARCHAR(20) NULL, department VARCHAR(80)
      NULL, "
                + "active BOOLEAN NOT NULL DEFAULT TRUE, "
                + "CHECK (role IN ('ADMIN','USER'))) ENGINE=InnoDB";
        String registrationAreasSql = "CREATE TABLE IF NOT EXISTS registration_areas ("
                + "area_name VARCHAR(100) PRIMARY KEY, assigned_username VARCHAR(40) UNIQUE NULL, "
                + "CONSTRAINT fk_registration_area_user FOREIGN KEY (assigned_username) "
                + "REFERENCES app_users(username) ON DELETE SET NULL) ENGINE=InnoDB";
        String wasteSql = "CREATE TABLE IF NOT EXISTS waste ("
                + "id INT AUTO_INCREMENT PRIMARY KEY, waste_type VARCHAR(30) NOT NULL, "
                + "description VARCHAR(255) NOT NULL, quantity DECIMAL(10,2) NOT NULL, "
                + "location VARCHAR(100) NOT NULL, status VARCHAR(20) NOT NULL DEFAULT 'PENDING', "
                + "created_by VARCHAR(40), collected_by VARCHAR(40), collected_at DATETIME NOT NULL, "
                + "invalid_reason VARCHAR(255), created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
       "
                + "updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
      "
                + "INDEX idx_waste_report (status, collected_at), "
                + "INDEX idx_waste_collector (collected_by, status), "
```

```
84                  + "INDEX idx_waste_area (location, status), "
85                  + "CONSTRAINT fk_waste_creator FOREIGN KEY (created_by) REFERENCES app_users(username),
      "
86                  + "CONSTRAINT fk_waste_collector FOREIGN KEY (collected_by) REFERENCES
      app_users(username), "
87                  + "CHECK (waste_type IN ('Organic','Recyclable','Hazardous')), "
88                  + "CHECK (status IN ('PENDING','COMPLETED','INVALID')), "
89                  + "CHECK (quantity > 0)) ENGINE=InnoDB";
90            String announcementSql = "CREATE TABLE IF NOT EXISTS announcements ("
91                  + "id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(120) NOT NULL, "
92                  + "message TEXT NOT NULL, created_by VARCHAR(40) NOT NULL, "
93                  + "created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP, "
94                  + "CONSTRAINT fk_announcement_author FOREIGN KEY (created_by) REFERENCES
      app_users(username)) ENGINE=InnoDB";
95            String readSql = "CREATE TABLE IF NOT EXISTS announcement_reads ("
96                  + "announcement_id INT NOT NULL, username VARCHAR(40) NOT NULL, "
97                  + "read_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP, "
98                  + "PRIMARY KEY (announcement_id, username), "
99                  + "CONSTRAINT fk_announcement_read_announcement FOREIGN KEY (announcement_id) REFERENCES
       announcements(id) ON DELETE CASCADE, "
100                 + "CONSTRAINT fk_announcement_read_user FOREIGN KEY (username) REFERENCES
      app_users(username) ON DELETE CASCADE) ENGINE=InnoDB";
101           String completionSql = "CREATE TABLE IF NOT EXISTS completion_requests ("
102                 + "id BIGINT AUTO_INCREMENT PRIMARY KEY, waste_id INT NOT NULL, requested_by VARCHAR(40)
       NOT NULL, "
103                 + "requested_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP, request_status VARCHAR(12)
       NOT NULL DEFAULT 'PENDING', "
104                 + "reviewed_by VARCHAR(40), reviewed_at TIMESTAMP NULL, "
105                 + "CONSTRAINT fk_completion_waste FOREIGN KEY (waste_id) REFERENCES waste(id) ON DELETE
      CASCADE, "
106                 + "CONSTRAINT fk_completion_requester FOREIGN KEY (requested_by) REFERENCES
      app_users(username), "
107                 + "CONSTRAINT fk_completion_reviewer FOREIGN KEY (reviewed_by) REFERENCES
      app_users(username), "
108                 + "CHECK (request_status IN ('PENDING','APPROVED','REJECTED','CANCELLED')), "
109                 + "INDEX idx_completion_status_requested (request_status, requested_at)) ENGINE=InnoDB";
110           try (Connection connection = getConnection(); Statement statement =
      connection.createStatement()) {
111                statement.executeUpdate(usersSql);
112                statement.executeUpdate(registrationAreasSql);
113                statement.executeUpdate(wasteSql);
114                statement.executeUpdate(announcementSql);
115                statement.executeUpdate(readSql);
116                statement.executeUpdate(completionSql);
117           }
118       }
119
120       private static void migrateUserTable() throws SQLException {
121           addUserColumnIfMissing("date_of_birth", "DATE NULL");
122           addUserColumnIfMissing("register_number", "VARCHAR(20) NULL");
123           addUserColumnIfMissing("department", "VARCHAR(80) NULL");
124           try (Connection connection = getConnection();
125                ResultSet indexes = connection.getMetaData().getIndexInfo(connection.getCatalog(), null,
126                        "app_users", true, false)) {
127                while (indexes.next()) {
128                    if ("uq_app_users_register_number".equalsIgnoreCase(indexes.getString("INDEX_NAME")))
      return;
129                }
130           }
131           try (Connection connection = getConnection(); Statement statement =
      connection.createStatement()) {
132                statement.executeUpdate("CREATE UNIQUE INDEX uq_app_users_register_number ON app_users
      (register_number)");
133           }
134       }
135
136       private static void addUserColumnIfMissing(String column, String definition) throws SQLException {
137           try (Connection connection = getConnection()) {
138                DatabaseMetaData metadata = connection.getMetaData();
139                try (ResultSet columns = metadata.getColumns(connection.getCatalog(), null, "app_users",
      column)) {
140                    if (columns.next()) return;
141                }
142                try (Statement statement = connection.createStatement()) {
143                    statement.executeUpdate("ALTER TABLE app_users ADD COLUMN " + column + " " +
      definition);
144                }
145           }
146       }
147
148       private static void seedRegistrationAreas() throws SQLException {
149           try (Connection connection = getConnection();
150                PreparedStatement statement = connection.prepareStatement(
151                        "INSERT IGNORE INTO registration_areas (area_name) VALUES (?)")) {
152                for (String area : REGISTRATION_AREAS) {
153                    statement.setString(1, area);
154                    statement.addBatch();
155                }
156                statement.executeBatch();
157           }
```

```
158     }
159
160     private static void migrateWasteTable() throws SQLException {
161         addColumnIfMissing("created_by", "VARCHAR(40) NULL");
162         addColumnIfMissing("collected_by", "VARCHAR(40) NULL");
163         addColumnIfMissing("collected_at", "DATETIME NULL");
164         addColumnIfMissing("invalid_reason", "VARCHAR(255) NULL");
165         addColumnIfMissing("created_at", "TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP");
166         addColumnIfMissing("updated_at", "TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE
          CURRENT_TIMESTAMP");
167         addIndexIfMissing("idx_waste_report", "status, collected_at");
168         addIndexIfMissing("idx_waste_collector", "collected_by, status");
169         addIndexIfMissing("idx_waste_area", "location, status");
170
171         try (Connection connection = getConnection(); Statement statement =
          connection.createStatement()) {
172             statement.executeUpdate("UPDATE waste SET collected_at = COALESCE(collected_at, created_at,
          CURRENT_TIMESTAMP)");
173             statement.executeUpdate("UPDATE waste SET status = CASE "
174                     + "WHEN UPPER(status) IN ('COLLECTED','COMPLETED') THEN 'COMPLETED' "
175                     + "WHEN UPPER(status) = 'INVALID' THEN 'INVALID' ELSE 'PENDING' END");
176         }
177     }
178
179     private static void addColumnIfMissing(String column, String definition) throws SQLException {
180         try (Connection connection = getConnection()) {
181             DatabaseMetaData metadata = connection.getMetaData();
182             try (ResultSet columns = metadata.getColumns(connection.getCatalog(), null, "waste",
          column)) {
183                 if (columns.next()) return;
184             }
185             try (Statement statement = connection.createStatement()) {
186                 statement.executeUpdate("ALTER TABLE waste ADD COLUMN " + column + " " + definition);
187             }
188         }
189     }
190
191     private static void addIndexIfMissing(String indexName, String columns) throws SQLException {
192         try (Connection connection = getConnection();
193              ResultSet indexes = connection.getMetaData().getIndexInfo(connection.getCatalog(), null,
194                      "waste", false, false)) {
195             while (indexes.next()) {
196                 if (indexName.equalsIgnoreCase(indexes.getString("INDEX_NAME"))) return;
197             }
198         }
199         try (Connection connection = getConnection(); Statement statement =
          connection.createStatement()) {
200             statement.executeUpdate("CREATE INDEX " + indexName + " ON waste (" + columns + ")");
201         }
202     }
203
204     private static void seedAccounts() throws SQLException {
205         String sql = "INSERT IGNORE INTO app_users "
206                 + "(username, display_name, assigned_area, role, password_salt, password_hash) VALUES
          (?, ?, ?, ?, ?, ?)";
207         try (Connection connection = getConnection(); PreparedStatement statement =
          connection.prepareStatement(sql)) {
208             if (!accountExists(connection, "deepthirani")) {
209                 String initialAdminPassword = setting("WASTE_ADMIN_INITIAL_PASSWORD", null);
210                 if (initialAdminPassword == null) {
211                     throw new SQLException("Set WASTE_ADMIN_INITIAL_PASSWORD before the first
          application launch.");
212                 }
213                 String adminSalt = PasswordHasher.newSalt();
214                 char[] adminPassword = initialAdminPassword.toCharArray();
215                 String adminHash = PasswordHasher.hash(adminPassword, adminSalt);
216                 java.util.Arrays.fill(adminPassword, '\0');
217                 statement.setString(1, "deepthirani");
218                 statement.setString(2, "Deepthi Rani");
219                 statement.setString(3, null);
220                 statement.setString(4, "ADMIN");
221                 statement.setString(5, adminSalt);
222                 statement.setString(6, adminHash);
223                 statement.executeUpdate();
224             }
225
226             for (String[] member : MEMBERS) {
227                 if (accountExists(connection, member[0])) continue;
228                 String salt = PasswordHasher.newSalt();
229                 char[] initialPassword = (member[0] + "_clean").toCharArray();
230                 String hash = PasswordHasher.hash(initialPassword, salt);
231                 java.util.Arrays.fill(initialPassword, '\0');
232                 statement.setString(1, member[0]);
233                 statement.setString(2, member[1]);
234                 statement.setString(3, member[2]);
235                 statement.setString(4, "USER");
236                 statement.setString(5, salt);
237                 statement.setString(6, hash);
238                 statement.executeUpdate();
239             }
```

```
240         }
241     }
242
243     private static boolean accountExists(Connection connection, String username) throws SQLException {
244         try (PreparedStatement statement = connection.prepareStatement("SELECT 1 FROM app_users WHERE
      username=?")) {
245             statement.setString(1, username);
246             try (ResultSet rows = statement.executeQuery()) {
247                 return rows.next();
248             }
249         }
250     }
251 }
```

B.11 PasswordHasher

```
  1 package dao;
  2
  3 import javax.crypto.SecretKeyFactory;
  4 import javax.crypto.spec.PBEKeySpec;
  5 import java.security.MessageDigest;
  6 import java.security.SecureRandom;
  7 import java.util.Locale;
  8
  9 public final class PasswordHasher {
 10     private static final int ITERATIONS = 310_000;
 11     private static final int KEY_BITS = 256;
 12
 13     private PasswordHasher() { }
 14
 15     public static String newSalt() {
 16         byte[] salt = new byte[16];
 17         new SecureRandom().nextBytes(salt);
 18         return toHex(salt);
 19     }
 20
 21     public static String hash(char[] password, String saltHex) {
 22         PBEKeySpec specification = new PBEKeySpec(password, fromHex(saltHex), ITERATIONS, KEY_BITS);
 23         try {
 24             byte[] result = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256")
 25                     .generateSecret(specification).getEncoded();
 26             return toHex(result);
 27         } catch (Exception exception) {
 28             throw new IllegalStateException("Password hashing is unavailable", exception);
 29         } finally {
 30             specification.clearPassword();
 31         }
 32     }
 33
 34     public static boolean matches(char[] password, String salt, String expectedHash) {
 35         byte[] actual = fromHex(hash(password, salt));
 36         byte[] expected = fromHex(expectedHash);
 37         return MessageDigest.isEqual(actual, expected);
 38     }
 39
 40     private static String toHex(byte[] bytes) {
 41         StringBuilder result = new StringBuilder(bytes.length * 2);
 42         for (byte value : bytes) result.append(String.format(Locale.ROOT, "%02x", value & 0xff));
 43         return result.toString();
 44     }
 45
 46     private static byte[] fromHex(String value) {
 47         byte[] result = new byte[value.length() / 2];
 48         for (int i = 0; i < result.length; i++) {
 49             result[i] = (byte) Integer.parseInt(value.substring(i * 2, i * 2 + 2), 16);
 50         }
 51         return result;
 52     }
 53 }
```

B.12 AuthenticationManager

```
package dao;

import model.AppUser;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class AuthenticationManager {
    public AppUser authenticate(String username, char[] password) throws SQLException {
        String sql = "SELECT username, display_name, assigned_area, role, date_of_birth,
      register_number, department, "
                + "password_salt, password_hash "
                + "FROM app_users WHERE username = ? AND active = TRUE";

        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, username.trim().toLowerCase());
            try (ResultSet result = statement.executeQuery()) {
                if (!result.next() || !PasswordHasher.matches(password,
                        result.getString("password_salt"), result.getString("password_hash"))) {
                    return null;
                }
                return new AppUser(result.getString("username"), result.getString("display_name"),
                        result.getString("assigned_area"), result.getString("role"),
                        result.getString("date_of_birth"), result.getString("register_number"),
                        result.getString("department"));
            }
        }
    }
}
```

B.13 UserManager

```
package dao;

import model.AppUser;
import model.RegistrationResult;

import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.security.SecureRandom;
import java.util.ArrayList;
import java.util.Collections;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import java.util.List;
import java.util.Locale;

public class UserManager {
    private static final SecureRandom RANDOM = new SecureRandom();

    public RegistrationResult register(String fullName, String dateOfBirth, String registerNumber,
                                       String department) throws SQLException {
        String name = normalizeName(fullName);
        String dobText = required(dateOfBirth, "Date of birth is required.");
        String register = required(registerNumber, "CET register number is
      required.").toUpperCase(Locale.ROOT);
        String course = required(department, "Department is required.");
        LocalDate dob;
        try {
            dob = LocalDate.parse(dobText);
        } catch (DateTimeParseException exception) {
            throw new IllegalArgumentException("Enter date of birth as yyyy-MM-dd.");
        }
        if (!dob.isBefore(LocalDate.now())) throw new IllegalArgumentException("Date of birth must be in
       the past.");
        if (register.length() > 20 || !register.matches("TVE\\d{2}[A-Z]{2,5}\\d{3}")) {
            throw new IllegalArgumentException("Use a CET register number such as TVE25CSE068.");
        }
        if (course.length() > 80) throw new IllegalArgumentException("Department name must be 80
      characters or fewer.");

        String firstName = name.split(" ")[0];
        if (!firstName.matches("[\\p{L}][\\p{L}\\p{N}'_-]{0,39}")) {
            throw new IllegalArgumentException("The first name must start with a letter and be 40
      characters or fewer.");
        }
        String username = firstName.toLowerCase(Locale.ROOT);
        String temporaryPassword = username + "_clean";
        String salt = PasswordHasher.newSalt();
        char[] password = temporaryPassword.toCharArray();
        String hash;
        try {
            hash = PasswordHasher.hash(password, salt);
        } finally {
            java.util.Arrays.fill(password, '\0');
        }

        try (Connection connection = DatabaseManager.getConnection()) {
            connection.setAutoCommit(false);
            try {
                ensureNotRegistered(connection, username, register);
                List<String> areas = lockUnassignedAreas(connection);
                if (areas.isEmpty()) throw new SQLException("All registration areas are assigned. Ask
      the administrator to add more areas.");
                Collections.shuffle(areas, RANDOM);
                String assignedArea = areas.get(0);

                String insert = "INSERT INTO app_users (username, display_name, date_of_birth,
      register_number, "
                        + "department, assigned_area, role, password_salt, password_hash) "
                        + "VALUES (?, ?, ?, ?, ?, ?, 'USER', ?, ?)";
                try (PreparedStatement statement = connection.prepareStatement(insert)) {
                    statement.setString(1, username);
                    statement.setString(2, name);
                    statement.setDate(3, Date.valueOf(dob));
                    statement.setString(4, register);
                    statement.setString(5, course);
                    statement.setString(6, assignedArea);
                    statement.setString(7, salt);
                    statement.setString(8, hash);
                    statement.executeUpdate();
                }
                try (PreparedStatement statement = connection.prepareStatement(
                        "UPDATE registration_areas SET assigned_username=? WHERE area_name=? AND
      assigned_username IS NULL")) {
                    statement.setString(1, username);
```

```
                    statement.setString(2, assignedArea);
                    if (statement.executeUpdate() != 1) throw new SQLException("Area assignment changed. Please try registration again.");
                }
                connection.commit();
                AppUser user = new AppUser(username, name, assignedArea, "USER", dob.toString(), register, course);
                return new RegistrationResult(user, temporaryPassword);
            } catch (SQLException | RuntimeException exception) {
                connection.rollback();
                if (exception instanceof SQLException) {
                    SQLException sqlException = (SQLException) exception;
                    if (sqlException.getSQLState() != null && sqlException.getSQLState().startsWith("23")) {
                        throw new SQLException("That username or CET register number is already registered.");
                    }
                }
                throw exception;
            }
        }
    }

    public List<Object[]> getMembers(AppUser actor, String keyword) throws SQLException {
        requireAdmin(actor);
        List<Object[]> members = new ArrayList<>();
        String sql = "SELECT display_name, username, COALESCE(DATE_FORMAT(date_of_birth,'%Y-%m-%d'),'Not provided'), "
                + "COALESCE(register_number,'Not provided'), COALESCE(department,'Not provided'), "
                + "COALESCE(assigned_area,'Not assigned'), IF(active,'Active','Inactive') "
                + "FROM app_users WHERE role='USER' AND (display_name LIKE ? OR username LIKE ? "
                + "OR COALESCE(register_number,'') LIKE ? OR COALESCE(department,'') LIKE ? "
                + "OR COALESCE(assigned_area,'') LIKE ?) ORDER BY display_name, username";
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            String match = "%" + (keyword == null ? "" : keyword.trim()) + "%";
            for (int i = 1; i <= 5; i++) statement.setString(i, match);
            try (ResultSet rows = statement.executeQuery()) {
                while (rows.next()) members.add(new Object[]{rows.getString(1), rows.getString(2), rows.getString(3),
                        rows.getString(4), rows.getString(5), rows.getString(6), rows.getString(7)});
            }
        }
        return members;
    }

    public void addRegistrationArea(String areaName, AppUser actor) throws SQLException {
        requireAdmin(actor);
        String area = required(areaName, "Area name is required.");
        if (area.length() > 100) throw new IllegalArgumentException("Area name must be 100 characters or fewer.");
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(
                     "INSERT INTO registration_areas (area_name) VALUES (?)")) {
            statement.setString(1, area);
            statement.executeUpdate();
        } catch (SQLException exception) {
            if (exception.getSQLState() != null && exception.getSQLState().startsWith("23")) {
                throw new SQLException("That registration area already exists.");
            }
            throw exception;
        }
    }

    private static String normalizeName(String value) {
        String name = required(value, "Full name is required.").replaceAll("\\s+", " ");
        if (name.length() > 100) {
            throw new IllegalArgumentException("Name must be 100 characters or fewer.");
        }
        return name;
    }

    private static String required(String value, String message) {
        String result = value == null ? "" : value.trim();
        if (result.isEmpty()) throw new IllegalArgumentException(message);
        return result;
    }

    private static List<String> lockUnassignedAreas(Connection connection) throws SQLException {
        List<String> areas = new ArrayList<>();
        try (PreparedStatement statement = connection.prepareStatement(
                "SELECT area_name FROM registration_areas WHERE assigned_username IS NULL FOR UPDATE");
             ResultSet rows = statement.executeQuery()) {
            while (rows.next()) areas.add(rows.getString(1));
        }
        return areas;
    }

    private static void ensureNotRegistered(Connection connection, String username, String register)
            throws SQLException {
```

```
163         try (PreparedStatement statement = connection.prepareStatement(
164                 "SELECT username, register_number FROM app_users WHERE username=? OR
          register_number=?")) {
165              statement.setString(1, username);
166              statement.setString(2, register);
167              try (ResultSet rows = statement.executeQuery()) {
168                  while (rows.next()) {
169                      if (username.equalsIgnoreCase(rows.getString("username"))) {
170                          throw new SQLException("The username '" + username + "' already exists. Ask the
          administrator for access.");
171                      }
172                      if (register.equalsIgnoreCase(rows.getString("register_number"))) {
173                          throw new SQLException("That CET register number is already registered.");
174                      }
175                  }
176              }
177         }
178     }
179
180     private static void requireAdmin(AppUser actor) throws SQLException {
181         if (actor == null || !actor.isAdmin()) throw new SQLException("Only an administrator can view
          the member directory.");
182     }
183 }
```

B.14 WasteManager

```
package dao;

import model.AppUser;
import model.HazardousWaste;
import model.OrganicWaste;
import model.RecyclableWaste;
import model.Waste;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.Timestamp;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;

public class WasteManager {
    public void addWaste(Waste waste, AppUser actor, String collectorUsername) throws SQLException {
        String collector = actor.isAdmin() ? emptyToNull(collectorUsername) : actor.getUsername();
        String sql = "INSERT INTO waste (waste_type, description, quantity, location, status,
      created_by, collected_by, collected_at) "
                + "VALUES (?, ?, ?, ?, 'PENDING', ?, ?, ?)";
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, waste.getWasteCategory());
            statement.setString(2, waste.getDescription().trim());
            statement.setDouble(3, waste.getQuantity());
            statement.setString(4, waste.getLocation());
            statement.setString(5, actor.getUsername());
            statement.setString(6, collector);
            statement.setTimestamp(7, Timestamp.valueOf(parseCollectedAt(waste.getCollectedAt())));
            statement.executeUpdate();
        }
    }

    public List<Waste> getVisibleWaste(AppUser actor, String keyword) throws SQLException {
        List<Waste> records = new ArrayList<>();
        String sql = "SELECT w.*, u.display_name AS collector_name, EXISTS (SELECT 1 FROM
      completion_requests cr "
                + "WHERE cr.waste_id=w.id AND cr.request_status='PENDING') AS completion_requested, "
                + "(SELECT cr.request_status FROM completion_requests cr WHERE cr.waste_id=w.id "
                + "ORDER BY cr.id DESC LIMIT 1) AS latest_request_status FROM waste w "
                + "LEFT JOIN app_users u ON u.username = w.collected_by WHERE "
                + (actor.isAdmin() ? "1=1" : "(w.created_by = ? OR w.collected_by = ?)")
                + " AND (w.description LIKE ? OR w.location LIKE ? OR w.waste_type LIKE ? OR w.status
      LIKE ? "
                + "OR COALESCE(u.display_name,'') LIKE ?) ORDER BY w.id DESC";
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            int parameter = 1;
            if (!actor.isAdmin()) {
                statement.setString(parameter++, actor.getUsername());
                statement.setString(parameter++, actor.getUsername());
            }
            String match = "%" + (keyword == null ? "" : keyword.trim()) + "%";
            for (int i = 0; i < 5; i++) statement.setString(parameter++, match);
            try (ResultSet rows = statement.executeQuery()) {
                while (rows.next()) records.add(createWaste(rows));
            }
        }
        return records;
    }

    public List<String[]> getCollectors() throws SQLException {
        List<String[]> users = new ArrayList<>();
        String sql = "SELECT username, display_name FROM app_users WHERE role='USER' AND active=TRUE
      ORDER BY display_name";
        try (Connection connection = DatabaseManager.getConnection();
             Statement statement = connection.createStatement();
             ResultSet rows = statement.executeQuery(sql)) {
            while (rows.next()) users.add(new String[]{rows.getString(1), rows.getString(2)});
        }
        return users;
    }

    public void updateWaste(Waste waste, AppUser actor, String collectorUsername) throws SQLException {
        String access = actor.isAdmin() ? "" : " AND (created_by = ? OR collected_by = ?)";
        String sql = "UPDATE waste SET waste_type=?, description=?, quantity=?, location=?,
      collected_at=?, collected_by=? "
                + "WHERE id=? AND status='PENDING'" + access;
        try (Connection connection = DatabaseManager.getConnection()) {
            connection.setAutoCommit(false);
            try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, waste.getWasteCategory());
```

```
            statement.setString(2, waste.getDescription().trim());
            statement.setDouble(3, waste.getQuantity());
            statement.setString(4, waste.getLocation());
            statement.setTimestamp(5, Timestamp.valueOf(parseCollectedAt(waste.getCollectedAt())));
            statement.setString(6, actor.isAdmin() ? emptyToNull(collectorUsername) :
    actor.getUsername());
            statement.setInt(7, waste.getId());
            if (!actor.isAdmin()) {
                statement.setString(8, actor.getUsername());
                statement.setString(9, actor.getUsername());
            }
                requireOneRow(statement.executeUpdate(), "Only your own pending records can be
    edited.");
            }
            cancelPendingRequests(connection, waste.getId());
            connection.commit();
        } catch (SQLException exception) {
            throw exception;
        }
    }

    public void markInvalid(int id, String reason, AppUser actor) throws SQLException {
        String sql = "UPDATE waste SET status='INVALID', invalid_reason=? WHERE id=? AND
    status='PENDING'"
                + (actor.isAdmin() ? "" : " AND (created_by=? OR collected_by=?)");
        try (Connection connection = DatabaseManager.getConnection()) {
            connection.setAutoCommit(false);
            try (PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, reason == null ? "" : reason.trim());
            statement.setInt(2, id);
            if (!actor.isAdmin()) {
                statement.setString(3, actor.getUsername());
                statement.setString(4, actor.getUsername());
            }
                requireOneRow(statement.executeUpdate(), "Only pending records you submitted can be
    marked invalid.");
            }
            cancelPendingRequests(connection, id);
            connection.commit();
        }
    }

    public void markCompleted(int id, AppUser actor) throws SQLException {
        if (!actor.isAdmin()) throw new SQLException("Only an administrator can mark a record
    completed.");
        try (Connection connection = DatabaseManager.getConnection()) {
            connection.setAutoCommit(false);
            try (PreparedStatement statement = connection.prepareStatement(
                    "UPDATE waste SET status='COMPLETED' WHERE id=? AND status='PENDING'")) {
            statement.setInt(1, id);
            requireOneRow(statement.executeUpdate(), "Only a pending record can be completed.");
            }
            cancelPendingRequests(connection, id);
            connection.commit();
        }
    }

    public void requestCompletion(int wasteId, AppUser actor) throws SQLException {
        if (actor.isAdmin()) throw new SQLException("Administrators can mark records collected
    directly.");
        try (Connection connection = DatabaseManager.getConnection()) {
            connection.setAutoCommit(false);
            try {
                String ownerSql = "SELECT status, created_by, collected_by FROM waste WHERE id=? FOR
    UPDATE";
                try (PreparedStatement statement = connection.prepareStatement(ownerSql)) {
                    statement.setInt(1, wasteId);
                    try (ResultSet row = statement.executeQuery()) {
                        if (!row.next() || !"PENDING".equals(row.getString("status"))) {
                            throw new SQLException("Only a pending record can be sent for approval.");
                        }
                        if (!actor.getUsername().equals(row.getString("created_by"))
                                && !actor.getUsername().equals(row.getString("collected_by"))) {
                            throw new SQLException("You can request completion only for your own
    records.");
                        }
                    }
                }
                try (PreparedStatement statement = connection.prepareStatement(
                        "SELECT 1 FROM completion_requests WHERE waste_id=? AND request_status='PENDING'
     FOR UPDATE")) {
                    statement.setInt(1, wasteId);
                    try (ResultSet row = statement.executeQuery()) {
                        if (row.next()) throw new SQLException("This record is already waiting for admin
     approval.");
                    }
                }
                try (PreparedStatement statement = connection.prepareStatement(
                        "INSERT INTO completion_requests (waste_id, requested_by) VALUES (?, ?)")) {
                    statement.setInt(1, wasteId);
```

```
163                     statement.setString(2, actor.getUsername());
164                     statement.executeUpdate();
165                 }
166                 connection.commit();
167             } catch (SQLException exception) {
168                 connection.rollback();
169                 throw exception;
170             }
171         }
172     }
173 
174     public int getPendingCompletionRequestCount(AppUser actor) throws SQLException {
175         requireAdmin(actor);
176         try (Connection connection = DatabaseManager.getConnection();
177              PreparedStatement statement = connection.prepareStatement(
178                      "SELECT COUNT(*) FROM completion_requests WHERE request_status='PENDING'")) {
179             try (ResultSet rows = statement.executeQuery()) {
180                 rows.next();
181                 return rows.getInt(1);
182             }
183         }
184     }
185 
186     public List<Object[]> getPendingCompletionRequests(AppUser actor) throws SQLException {
187         requireAdmin(actor);
188         List<Object[]> requests = new ArrayList<>();
189         String sql = "SELECT cr.id, w.id, w.waste_type, w.description, w.quantity, w.location,
      w.collected_at, "
190                 + "u.display_name, cr.requested_at FROM completion_requests cr "
191                 + "JOIN waste w ON w.id=cr.waste_id JOIN app_users u ON u.username=cr.requested_by "
192                 + "WHERE cr.request_status='PENDING' AND w.status='PENDING' ORDER BY cr.requested_at,
      cr.id";
193         try (Connection connection = DatabaseManager.getConnection();
194              PreparedStatement statement = connection.prepareStatement(sql);
195              ResultSet rows = statement.executeQuery()) {
196             while (rows.next()) requests.add(new Object[]{rows.getLong(1), rows.getInt(2),
      rows.getString(3),
197                     rows.getString(4), rows.getDouble(5), rows.getString(6),
      rows.getTimestamp(7).toLocalDateTime()
198                     .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")), rows.getString(8),
199                     rows.getTimestamp(9).toLocalDateTime().format(DateTimeFormatter.ofPattern("yyyy-MM-
      dd HH:mm"))});
200         }
201         return requests;
202     }
203 
204     public void approveCompletionRequest(long requestId, AppUser actor) throws SQLException {
205         requireAdmin(actor);
206         updateCompletionRequest(requestId, actor, true);
207     }
208 
209     public void rejectCompletionRequest(long requestId, AppUser actor) throws SQLException {
210         requireAdmin(actor);
211         updateCompletionRequest(requestId, actor, false);
212     }
213 
214     private void updateCompletionRequest(long requestId, AppUser actor, boolean approve) throws
      SQLException {
215         try (Connection connection = DatabaseManager.getConnection()) {
216             connection.setAutoCommit(false);
217             try {
218                 int wasteId;
219                 try (PreparedStatement statement = connection.prepareStatement(
220                         "SELECT waste_id FROM completion_requests WHERE id=? AND
      request_status='PENDING' FOR UPDATE")) {
221                     statement.setLong(1, requestId);
222                     try (ResultSet row = statement.executeQuery()) {
223                         if (!row.next()) throw new SQLException("This approval request is no longer
      pending.");
224                         wasteId = row.getInt(1);
225                     }
226                 }
227                 if (approve) {
228                     try (PreparedStatement statement = connection.prepareStatement(
229                             "UPDATE waste SET status='COMPLETED' WHERE id=? AND status='PENDING'")) {
230                         statement.setInt(1, wasteId);
231                         requireOneRow(statement.executeUpdate(), "The waste record is no longer
      pending.");
232                     }
233                 }
234                 try (PreparedStatement statement = connection.prepareStatement(
235                         "UPDATE completion_requests SET request_status=?, reviewed_by=?,
      reviewed_at=CURRENT_TIMESTAMP "
236                                 + "WHERE id=? AND request_status='PENDING'")) {
237                     statement.setString(1, approve ? "APPROVED" : "REJECTED");
238                     statement.setString(2, actor.getUsername());
239                     statement.setLong(3, requestId);
240                     requireOneRow(statement.executeUpdate(), "This approval request was already
      reviewed.");
241                 }
```

```
242                 connection.commit();
243             } catch (SQLException exception) {
244                 connection.rollback();
245                 throw exception;
246             }
247         }
248     }
249
250     public void deleteWaste(int id, AppUser actor) throws SQLException {
251         if (!actor.isAdmin()) throw new SQLException("Only an administrator can delete records.");
252         try (Connection connection = DatabaseManager.getConnection();
253              PreparedStatement statement = connection.prepareStatement("DELETE FROM waste WHERE id=?"))
          {
254             statement.setInt(1, id);
255             requireOneRow(statement.executeUpdate(), "Record was not found.");
256         }
257     }
258
259     private Waste createWaste(ResultSet rows) throws SQLException {
260         int id = rows.getInt("id");
261         String type = rows.getString("waste_type");
262         String description = rows.getString("description");
263         double quantity = rows.getDouble("quantity");
264         String location = rows.getString("location");
265         String status = rows.getString("status");
266         Waste waste;
267         if ("Organic".equalsIgnoreCase(type)) waste = new OrganicWaste(id, description, quantity,
          location, status);
268         else if ("Recyclable".equalsIgnoreCase(type)) waste = new RecyclableWaste(id, description,
          quantity, location, status);
269         else waste = new HazardousWaste(id, description, quantity, location, status);
270         waste.setCollectedBy(rows.getString("collected_by"));
271         waste.setCreatedBy(rows.getString("created_by"));
272         Timestamp timestamp = rows.getTimestamp("collected_at");
273         waste.setCollectedAt(timestamp == null ? "" : timestamp.toLocalDateTime()
274                 .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")));
275         waste.setInvalidReason(rows.getString("invalid_reason"));
276         waste.setCompletionRequested(rows.getBoolean("completion_requested"));
277         waste.setCompletionRequestStatus(rows.getString("latest_request_status"));
278         return waste;
279     }
280
281     private static void cancelPendingRequests(Connection connection, int wasteId) throws SQLException {
282         try (PreparedStatement statement = connection.prepareStatement(
283                 "UPDATE completion_requests SET request_status='CANCELLED' WHERE waste_id=? AND
          request_status='PENDING'")) {
284             statement.setInt(1, wasteId);
285             statement.executeUpdate();
286         }
287     }
288
289     private static void requireAdmin(AppUser actor) throws SQLException {
290         if (!actor.isAdmin()) throw new SQLException("Only an administrator can review completion
          requests.");
291     }
292
293     private static String emptyToNull(String value) {
294         return value == null || value.trim().isEmpty() ? null : value;
295     }
296
297     private static LocalDateTime parseCollectedAt(String value) {
298         String input = value.trim();
299         if (input.indexOf('T') >= 0) return LocalDateTime.parse(input);
300         return LocalDateTime.parse(input, DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm"));
301     }
302
303     private static void requireOneRow(int count, String message) throws SQLException {
304         if (count != 1) throw new SQLException(message);
305     }
306 }
```

B.15 ReportManager

```
package dao;

import model.AppUser;
import model.ReportRow;
import model.ReportSummary;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Timestamp;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class ReportManager {
    public ReportSummary getSummary(AppUser user, LocalDate from, LocalDate to) throws SQLException {
        String sql = "SELECT COALESCE(SUM(quantity),0), "
                + "COALESCE(SUM(CASE WHEN waste_type='Organic' THEN quantity ELSE 0 END),0), "
                + "COALESCE(SUM(CASE WHEN waste_type='Recyclable' THEN quantity ELSE 0 END),0), "
                + "COALESCE(SUM(CASE WHEN waste_type='Hazardous' THEN quantity ELSE 0 END),0), COUNT(*)
      "
                + "FROM waste WHERE status='COMPLETED' AND collected_at>=? AND collected_at<?"
                + (user.isAdmin() ? "" : " AND collected_by=?");
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            bindDateRange(statement, from, to, 1);
            if (!user.isAdmin()) statement.setString(3, user.getUsername());
            try (ResultSet result = statement.executeQuery()) {
                result.next();
                return new ReportSummary(result.getDouble(1), result.getDouble(2), result.getDouble(3),
                        result.getDouble(4), result.getInt(5));
            }
        }
    }

    public ReportRow getUserReport(String username, LocalDate from, LocalDate to, AppUser actor) throws
      SQLException {
        if (!actor.isAdmin() && !actor.getUsername().equals(username)) {
            throw new SQLException("Members can only view their own report.");
        }
        String sql = "SELECT u.display_name, "
                + "COALESCE(SUM(CASE WHEN w.waste_type='Organic' THEN w.quantity ELSE 0 END),0), "
                + "COALESCE(SUM(CASE WHEN w.waste_type='Recyclable' THEN w.quantity ELSE 0 END),0), "
                + "COALESCE(SUM(CASE WHEN w.waste_type='Hazardous' THEN w.quantity ELSE 0 END),0),
      COUNT(w.id), "
                + "COALESCE(SUM(CASE WHEN LOWER(w.location)=LOWER(u.assigned_area) THEN w.quantity ELSE
      0 END),0), "
                + "COALESCE(SUM(CASE WHEN LOWER(w.location)<>LOWER(u.assigned_area) THEN w.quantity ELSE
       0 END),0) "
                + "FROM app_users u LEFT JOIN waste w ON w.collected_by=u.username AND
      w.status='COMPLETED' "
                + "AND w.collected_at>=? AND w.collected_at<? WHERE u.username=? AND u.role='USER' "
                + "GROUP BY u.username,u.display_name";
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            bindDateRange(statement, from, to, 1);
            statement.setString(3, username);
            try (ResultSet result = statement.executeQuery()) {
                if (!result.next()) throw new SQLException("User report was not found.");
                return new ReportRow(result.getString(1), result.getDouble(2), result.getDouble(3),
                        result.getDouble(4), result.getInt(5), result.getDouble(6),
      result.getDouble(7));
            }
        }
    }

    public List<ReportRow> getAllUserReports(LocalDate from, LocalDate to, AppUser actor) throws
      SQLException {
        requireAdmin(actor);
        List<ReportRow> reports = new ArrayList<>();
        String sql = "SELECT u.display_name, "
                + "COALESCE(SUM(CASE WHEN w.waste_type='Organic' THEN w.quantity ELSE 0 END),0), "
                + "COALESCE(SUM(CASE WHEN w.waste_type='Recyclable' THEN w.quantity ELSE 0 END),0), "
                + "COALESCE(SUM(CASE WHEN w.waste_type='Hazardous' THEN w.quantity ELSE 0 END),0),
      COUNT(w.id), "
                + "COALESCE(SUM(CASE WHEN LOWER(w.location)=LOWER(u.assigned_area) THEN w.quantity ELSE
      0 END),0), "
                + "COALESCE(SUM(CASE WHEN LOWER(w.location)<>LOWER(u.assigned_area) THEN w.quantity ELSE
       0 END),0) "
                + "FROM app_users u LEFT JOIN waste w ON w.collected_by=u.username AND
      w.status='COMPLETED' "
                + "AND w.collected_at>=? AND w.collected_at<? WHERE u.role='USER' AND u.active=TRUE "
                + "GROUP BY u.username,u.display_name ORDER BY u.display_name";
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            bindDateRange(statement, from, to, 1);
```

```
            try (ResultSet result = statement.executeQuery()) {
                while (result.next()) reports.add(new ReportRow(result.getString(1),
      result.getDouble(2),
                        result.getDouble(3), result.getDouble(4), result.getInt(5),
                        result.getDouble(6), result.getDouble(7)));
            }
        }
        return reports;
    }

    public List<ReportRow> getAreaReports(LocalDate from, LocalDate to, AppUser actor) throws
      SQLException {
        requireAdmin(actor);
        List<ReportRow> reports = new ArrayList<>();
        String sql = "SELECT location, "
                + "COALESCE(SUM(CASE WHEN waste_type='Organic' THEN quantity ELSE 0 END),0), "
                + "COALESCE(SUM(CASE WHEN waste_type='Recyclable' THEN quantity ELSE 0 END),0), "
                + "COALESCE(SUM(CASE WHEN waste_type='Hazardous' THEN quantity ELSE 0 END),0), COUNT(*)
      "
                + "FROM waste WHERE status='COMPLETED' AND collected_at>=? AND collected_at<? "
                + "GROUP BY location ORDER BY SUM(quantity) DESC";
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            bindDateRange(statement, from, to, 1);
            try (ResultSet result = statement.executeQuery()) {
                while (result.next()) reports.add(new ReportRow(result.getString(1),
      result.getDouble(2),
                        result.getDouble(3), result.getDouble(4), result.getInt(5), 0, 0));
            }
        }
        return reports;
    }

    private static void requireAdmin(AppUser actor) throws SQLException {
        if (!actor.isAdmin()) throw new SQLException("Only an administrator can view organization-wide
      reports.");
    }

    private static void bindDateRange(PreparedStatement statement, LocalDate from, LocalDate to, int
      firstIndex)
            throws SQLException {
        if (from == null || to == null || from.isAfter(to)) {
            throw new IllegalArgumentException("A valid report date range is required.");
        }
        statement.setTimestamp(firstIndex, Timestamp.valueOf(from.atStartOfDay()));
        statement.setTimestamp(firstIndex + 1, Timestamp.valueOf(to.plusDays(1).atStartOfDay()));
    }
}
```

B.16 AnnouncementManager

```
package dao;

import model.Announcement;
import model.AppUser;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Timestamp;
import java.util.ArrayList;
import java.util.List;

public class AnnouncementManager {
    public void publish(String title, String message, AppUser actor) throws SQLException {
        if (!actor.isAdmin()) throw new SQLException("Only an administrator can publish
      announcements.");
        String cleanTitle = title == null ? "" : title.trim();
        String cleanMessage = message == null ? "" : message.trim();
        if (cleanTitle.isEmpty() || cleanMessage.isEmpty()) {
            throw new SQLException("Announcement title and message are required.");
        }
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(
                     "INSERT INTO announcements (title, message, created_by) VALUES (?, ?, ?)")) {
            statement.setString(1, cleanTitle);
            statement.setString(2, cleanMessage);
            statement.setString(3, actor.getUsername());
            statement.executeUpdate();
        }
    }

    public List<Announcement> getAnnouncements(AppUser user) throws SQLException {
        String sql = "SELECT a.id, a.title, a.message, u.display_name, a.created_at, "
                + "CASE WHEN r.username IS NULL THEN FALSE ELSE TRUE END AS is_read "
                + "FROM announcements a JOIN app_users u ON u.username=a.created_by "
                + "LEFT JOIN announcement_reads r ON r.announcement_id=a.id AND r.username=? "
                + "ORDER BY a.created_at DESC, a.id DESC";
        List<Announcement> announcements = new ArrayList<>();
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, user.getUsername());
            try (ResultSet rows = statement.executeQuery()) {
                while (rows.next()) {
                    Timestamp timestamp = rows.getTimestamp("created_at");
                    announcements.add(new Announcement(rows.getInt("id"), rows.getString("title"),
                            rows.getString("message"), rows.getString("display_name"),
                            timestamp.toLocalDateTime(), rows.getBoolean("is_read")));
                }
            }
        }
        return announcements;
    }

    public int getUnreadCount(AppUser user) throws SQLException {
        if (user.isAdmin()) return 0;
        String sql = "SELECT COUNT(*) FROM announcements a LEFT JOIN announcement_reads r "
                + "ON r.announcement_id=a.id AND r.username=? WHERE r.username IS NULL";
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(sql)) {
            statement.setString(1, user.getUsername());
            try (ResultSet rows = statement.executeQuery()) {
                rows.next();
                return rows.getInt(1);
            }
        }
    }

    public void markRead(int announcementId, AppUser user) throws SQLException {
        try (Connection connection = DatabaseManager.getConnection();
             PreparedStatement statement = connection.prepareStatement(
                     "INSERT IGNORE INTO announcement_reads (announcement_id, username) "
                             + "SELECT id, ? FROM announcements WHERE id=?")) {
            statement.setString(1, user.getUsername());
            statement.setInt(2, announcementId);
            statement.executeUpdate();
        }
    }
}
```

B.17 Announcement model

```
package model;

import java.time.LocalDateTime;

public class Announcement {
    private final int id;
    private final String title;
    private final String message;
    private final String author;
    private final LocalDateTime createdAt;
    private final boolean read;

    public Announcement(int id, String title, String message, String author,
                        LocalDateTime createdAt, boolean read) {
        this.id = id;
        this.title = title;
        this.message = message;
        this.author = author;
        this.createdAt = createdAt;
        this.read = read;
    }

    public int getId() { return id; }
    public String getTitle() { return title; }
    public String getMessage() { return message; }
    public String getAuthor() { return author; }
    public LocalDateTime getCreatedAt() { return createdAt; }
    public boolean isRead() { return read; }
}
```

B.18 LoginFrame

```
package gui;

import dao.AuthenticationManager;
import dao.UserManager;
import model.AppUser;
import model.RegistrationResult;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.border.EmptyBorder;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Font;
import java.awt.GridLayout;
import java.sql.SQLException;

public class LoginFrame extends JFrame {
    private final AuthenticationManager authenticationManager = new AuthenticationManager();
    private final UserManager userManager = new UserManager();
    private final JTextField usernameField = new JTextField();
    private final JPasswordField passwordField = new JPasswordField();

    public LoginFrame() {
        setTitle("Waste Management System | Sign in");
        setSize(480, 450);
        setResizable(false);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        JPanel page = new JPanel(new BorderLayout(0, 18));
        page.setBackground(new Color(246, 246, 250));
        page.setBorder(new EmptyBorder(25, 30, 28, 30));

        JPanel brand = new JPanel(new GridLayout(2, 1, 0, 7));
        brand.setOpaque(false);
        JLabel title = new JLabel("Waste Management", SwingConstants.CENTER);
        title.setFont(new Font("SansSerif", Font.BOLD, 25));
        title.setForeground(new Color(48, 42, 105));
        JLabel subtitle = new JLabel("Sign in to your Clean CET workspace", SwingConstants.CENTER);
        subtitle.setForeground(new Color(103, 103, 119));
        brand.add(title);
        brand.add(subtitle);

        JPanel form = new JPanel(new GridLayout(4, 1, 0, 8));
        form.setOpaque(false);
        form.add(label("Username"));
        form.add(usernameField);
        form.add(label("Password"));
        form.add(passwordField);
        styleInput(usernameField);
        styleInput(passwordField);

        JButton signIn = new JButton("Sign in");
        signIn.setUI(new CleanButtonUI());
        signIn.setOpaque(true);
        signIn.setContentAreaFilled(true);
        signIn.setBorderPainted(true);
        signIn.setBackground(new Color(89, 70, 168));
        signIn.setForeground(Color.WHITE);
        signIn.setFocusPainted(false);
        signIn.setBorder(new EmptyBorder(12, 15, 12, 15));
        signIn.addActionListener(event -> signIn());
        passwordField.addActionListener(event -> signIn());

        page.add(brand, BorderLayout.NORTH);
        page.add(form, BorderLayout.CENTER);
        JButton register = new JButton("Join Clean CET");
        register.setUI(new CleanButtonUI());
        register.setOpaque(true);
        register.setContentAreaFilled(true);
        register.setBackground(new Color(246, 244, 251));
        register.setForeground(new Color(54, 42, 112));
        register.setFocusPainted(false);
        register.setBorder(BorderFactory.createCompoundBorder(
                BorderFactory.createLineBorder(new Color(215, 215, 226)), new EmptyBorder(10, 15, 10,
      15)));
        register.addActionListener(event -> registerMember());
        JPanel actions = new JPanel(new GridLayout(2, 1, 0, 8));
        actions.setOpaque(false);
        actions.add(signIn);
```

```
87          actions.add(register);
88          page.add(actions, BorderLayout.SOUTH);
89          setContentPane(page);
90      }
91
92      private void registerMember() {
93          JTextField name = new JTextField();
94          JTextField dateOfBirth = new JTextField();
95          JTextField registerNumber = new JTextField();
96          JTextField department = new JTextField();
97          styleInput(name);
98          styleInput(dateOfBirth);
99          styleInput(registerNumber);
100         styleInput(department);
101
102         JPanel fields = new JPanel(new GridLayout(4, 1, 0, 8));
103         fields.add(labeled("Full name", name));
104         fields.add(labeled("Date of birth (yyyy-MM-dd)", dateOfBirth));
105         fields.add(labeled("CET register number (example: TVE25CSE068)", registerNumber));
106         fields.add(labeled("Department", department));
107         fields.setPreferredSize(new java.awt.Dimension(430, 340));
108         if (JOptionPane.showConfirmDialog(this, fields, "Join Clean CET", JOptionPane.OK_CANCEL_OPTION,
109                 JOptionPane.PLAIN_MESSAGE) != JOptionPane.OK_OPTION) return;
110
111         try {
112             RegistrationResult registration = userManager.register(name.getText(),
        dateOfBirth.getText(),
113                     registerNumber.getText(), department.getText());
114             AppUser newUser = registration.getUser();
115             String message = "Your account is ready.\n\n"
116                     + "Name: " + newUser.getDisplayName() + "\n"
117                     + "Username: " + newUser.getUsername() + "\n"
118                     + "Temporary password: " + registration.getTemporaryPassword() + "\n"
119                     + "Assigned area: " + newUser.getAssignedArea() + "\n\n"
120                     + "This password follows the requested username_clean pattern and is predictable. "
121                     + "Use it only for this classroom project. Your password is stored as a salted
        hash.";
122             JOptionPane.showMessageDialog(this, message, "Registration complete",
        JOptionPane.INFORMATION_MESSAGE);
123             dispose();
124             new WasteManagementGUI(newUser).setVisible(true);
125         } catch (IllegalArgumentException | SQLException exception) {
126             JOptionPane.showMessageDialog(this, exception.getMessage(), "Registration could not be
        completed",
127                     JOptionPane.ERROR_MESSAGE);
128         }
129     }
130
131     private void signIn() {
132         char[] password = passwordField.getPassword();
133         try {
134             AppUser user = authenticationManager.authenticate(usernameField.getText(), password);
135             if (user == null) {
136                 passwordField.setText("");
137                 JOptionPane.showMessageDialog(this, "Username or password is incorrect.", "Sign-in
        failed",
138                         JOptionPane.ERROR_MESSAGE);
139                 return;
140             }
141             dispose();
142             new WasteManagementGUI(user).setVisible(true);
143         } catch (SQLException exception) {
144             JOptionPane.showMessageDialog(this, "Could not access the database: " +
        exception.getMessage(),
145                     "Database error", JOptionPane.ERROR_MESSAGE);
146         } finally {
147             java.util.Arrays.fill(password, '\0');
148         }
149     }
150
151     private JLabel label(String text) {
152         JLabel label = new JLabel(text);
153         label.setForeground(new Color(58, 57, 72));
154         return label;
155     }
156
157     private JPanel labeled(String text, JTextField field) {
158         JPanel wrapper = new JPanel(new BorderLayout(0, 4));
159         wrapper.setOpaque(false);
160         JLabel title = label(text);
161         wrapper.add(title, BorderLayout.NORTH);
162         wrapper.add(field, BorderLayout.CENTER);
163         return wrapper;
164     }
165
166     private void styleInput(JTextField field) {
167         field.setFont(new Font("SansSerif", Font.PLAIN, 14));
168         field.setBorder(BorderFactory.createCompoundBorder(
169                 BorderFactory.createLineBorder(new Color(215, 215, 226)),
170                 new EmptyBorder(8, 10, 8, 10)));
```

```
171     }
172 }
```

B.19 CleanButtonUI

```
  1 package gui;
  2
  3 import javax.swing.AbstractButton;
  4 import javax.swing.JComponent;
  5 import javax.swing.plaf.basic.BasicButtonUI;
  6 import java.awt.Color;
  7 import java.awt.Graphics;
  8
  9 /** Paints button fills consistently across native Swing look-and-feels. */
 10 public class CleanButtonUI extends BasicButtonUI {
 11     @Override
 12     public void paint(Graphics graphics, JComponent component) {
 13         AbstractButton button = (AbstractButton) component;
 14         Color fill = button.getBackground();
 15         if (!button.isEnabled()) fill = new Color(fill.getRed(), fill.getGreen(), fill.getBlue(), 120);
 16         else if (button.getModel().isPressed()) fill = fill.darker();
 17         else if (button.getModel().isRollover()) fill = fill.brighter();
 18
 19         graphics.setColor(fill);
 20         graphics.fillRect(0, 0, component.getWidth(), component.getHeight());
 21         super.paint(graphics, component);
 22     }
 23 }
```

B.20 WasteManagementGUI

```
package gui;

import dao.ReportManager;
import dao.UserManager;
import dao.AnnouncementManager;
import dao.WasteManager;
import model.Announcement;
import model.AppUser;
import model.ReportRow;
import model.ReportSummary;
import model.Waste;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFileChooser;
import javax.swing.filechooser.FileNameExtensionFilter;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextArea;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTabbedPane;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.ListSelectionModel;
import javax.swing.SwingConstants;
import javax.swing.Timer;
import javax.swing.border.EmptyBorder;
import javax.swing.table.DefaultTableModel;
import java.awt.BorderLayout;
import java.awt.CardLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.FlowLayout;
import java.awt.GridLayout;
import java.awt.Insets;
import java.io.File;
import java.io.BufferedWriter;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.sql.SQLException;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;
import java.util.ArrayList;
import java.util.List;

public class WasteManagementGUI extends JFrame {
    private static final Color PURPLE = new Color(82, 63, 153);
    private static final Color PURPLE_DARK = new Color(54, 42, 112);
    private static final Color PURPLE_LIGHT = new Color(239, 235, 251);
    private static final Color PAGE = new Color(246, 246, 249);
    private static final Color INK = new Color(40, 40, 52);
    private static final Color MUTED = new Color(111, 111, 126);
    private static final Color LINE = new Color(226, 225, 234);
    private static final String[] AREAS = {
            "Civil canteen", "Mech front", "Dhwani", "Gazebo", "CS front", "Golden walkway",
            "Electrical gate", "Indoor court", "Archie corner", "College front", "DJ hall", "CGPU hall",
       "CETAA hall",
            "Industrial department front", "EC department front", "Rohith square", "Sargam stage",
      "Library front",
            "Ohm's villa", "Industrial base", "College ground", "Research block", "Old EC department",
            "Hydraulics lab front", "Back gate", "Mh front", "Lh front", "Flag post"
    };
    private static final DateTimeFormatter DATE_TIME_FORMAT = DateTimeFormatter.ofPattern("yyyy-MM-dd
      HH:mm");

    private final AppUser user;
    private final WasteManager wasteManager = new WasteManager();
    private final ReportManager reportManager = new ReportManager();
    private final AnnouncementManager announcementManager = new AnnouncementManager();
    private final UserManager userManager = new UserManager();
    private Timer approvalRefreshTimer;
    private final CardLayout pageLayout = new CardLayout();
    private final JPanel pageHost = new JPanel(pageLayout);
    private JButton announcementsNavigation;
    private JButton completionRequestsNavigation;
    private JTable announcementsTable;
    private DefaultTableModel announcementsModel;
    private JTextArea announcementDetails;
    private List<Announcement> announcements = new ArrayList<>();
    private JTable completionRequestsTable;
    private DefaultTableModel completionRequestsModel;
```

```
    private JTextField memberSearchField;
    private DefaultTableModel memberDirectoryModel;

    private JTextField descriptionField;
    private JTextField quantityField;
    private JTextField collectedAtField;
    private JComboBox<String> typeCombo;
    private JComboBox<String> areaCombo;
    private JComboBox<String> collectorCombo;
    private JTextField searchField;
    private JTable recordsTable;
    private DefaultTableModel recordsModel;
    private int selectedId = -1;
    private String selectedStatus;

    private JTextField fromField;
    private JTextField toField;
    private JLabel totalValue;
    private JLabel organicValue;
    private JLabel recyclableValue;
    private JLabel hazardousValue;
    private JLabel recordCountValue;
    private JLabel appliedRangeLabel;
    private JTable memberReportTable;
    private DefaultTableModel memberReportModel;
    private JTable areaReportTable;
    private DefaultTableModel areaReportModel;
    private JTable personalReportTable;
    private DefaultTableModel personalReportModel;
    private AreaChart areaChart;
    private List<ReportRow> currentMemberReports = new ArrayList<>();
    private List<ReportRow> currentAreaReports = new ArrayList<>();
    private LocalDate appliedFrom;
    private LocalDate appliedTo;

    public WasteManagementGUI(AppUser user) {
        this.user = user;
        setTitle("Clean CET | Waste Management");
        setSize(1240, 790);
        setMinimumSize(new Dimension(1050, 680));
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        getContentPane().setBackground(PAGE);

        pageHost.setBackground(PAGE);
        pageHost.add(createRecordsPage(), "records");
        pageHost.add(createReportsPage(), "reports");
        pageHost.add(createAnnouncementsPage(), "announcements");
        if (user.isAdmin()) {
            pageHost.add(createApprovalsPage(), "approvals");
            pageHost.add(createMembersPage(), "members");
        }
        add(createSidebar(), BorderLayout.WEST);
        add(createTopBar(), BorderLayout.NORTH);
        add(pageHost, BorderLayout.CENTER);

        refreshRecords();
        refreshReports();
        refreshAnnouncements();
        refreshApprovalRequests();
        refreshMembers();
        if (user.isAdmin()) {
            approvalRefreshTimer = new Timer(15000, event -> refreshApprovalCount());
            approvalRefreshTimer.start();
        }
    }

    private JPanel createSidebar() {
        JPanel sidebar = new JPanel(new BorderLayout(0, 22));
        sidebar.setBackground(PURPLE_DARK);
        sidebar.setPreferredSize(new Dimension(205, 0));
        sidebar.setBorder(new EmptyBorder(24, 15, 20, 15));

        JLabel brand = new JLabel("CLEAN CET", SwingConstants.CENTER);
        brand.setFont(new Font("SansSerif", Font.BOLD, 19));
        brand.setForeground(Color.WHITE);

        JPanel navigation = new JPanel(new GridLayout(0, 1, 0, 8));
        navigation.setOpaque(false);
        JButton records = navButton("Waste records");
        JButton reports = navButton(user.isAdmin() ? "Reports dashboard" : "My progress");
        JButton members = user.isAdmin() ? navButton("Member directory") : null;
        announcementsNavigation = navButton("Announcements");
        if (user.isAdmin()) completionRequestsNavigation = navButton("Collection approvals (0)");
        records.addActionListener(event -> pageLayout.show(pageHost, "records"));
        reports.addActionListener(event -> {
            refreshReports();
            pageLayout.show(pageHost, "reports");
        });
        if (user.isAdmin()) members.addActionListener(event -> {
```

```
175             refreshMembers();
176             pageLayout.show(pageHost, "members");
177         });
178         announcementsNavigation.addActionListener(event -> {
179             refreshAnnouncements();
180             pageLayout.show(pageHost, "announcements");
181         });
182         if (user.isAdmin()) completionRequestsNavigation.addActionListener(event -> {
183             refreshApprovalRequests();
184             pageLayout.show(pageHost, "approvals");
185         });
186         navigation.add(records);
187         navigation.add(reports);
188         if (user.isAdmin()) navigation.add(members);
189         navigation.add(announcementsNavigation);
190         if (user.isAdmin()) navigation.add(completionRequestsNavigation);
191 
192         JLabel footer = new JLabel("Waste collection tracker", SwingConstants.CENTER);
193         footer.setForeground(new Color(211, 205, 235));
194         footer.setFont(new Font("SansSerif", Font.PLAIN, 11));
195         sidebar.add(brand, BorderLayout.NORTH);
196         sidebar.add(navigation, BorderLayout.CENTER);
197         sidebar.add(footer, BorderLayout.SOUTH);
198         return sidebar;
199     }
200 
201     private JButton navButton(String text) {
202         JButton button = new JButton(text);
203         button.setUI(new CleanButtonUI());
204         button.setOpaque(true);
205         button.setContentAreaFilled(true);
206         button.setBorderPainted(false);
207         button.setHorizontalAlignment(SwingConstants.LEFT);
208         button.setForeground(Color.WHITE);
209         button.setBackground(PURPLE);
210         button.setFont(new Font("SansSerif", Font.BOLD, 13));
211         button.setFocusPainted(false);
212         button.setBorder(new EmptyBorder(12, 12, 12, 10));
213         return button;
214     }
215 
216     private JPanel createTopBar() {
217         JPanel top = new JPanel(new BorderLayout());
218         top.setBackground(Color.WHITE);
219         top.setBorder(BorderFactory.createCompoundBorder(BorderFactory.createMatteBorder(0, 0, 1, 0,
        LINE),
220                 new EmptyBorder(15, 22, 15, 24)));
221         JLabel title = new JLabel("Waste collection workspace");
222         title.setFont(new Font("SansSerif", Font.BOLD, 17));
223         title.setForeground(INK);
224         String identity = user.isAdmin() ? user.getDisplayName() + "  |  Administrator"
225                 : user.getDisplayName() + "  |  " + user.getAssignedArea();
226         JLabel profile = new JLabel(identity);
227         profile.setForeground(MUTED);
228         JPanel profileControls = new JPanel(new FlowLayout(FlowLayout.RIGHT, 12, 0));
229         profileControls.setOpaque(false);
230         JButton logout = button("Log out", false);
231         logout.addActionListener(event -> logout());
232         profileControls.add(profile);
233         profileControls.add(logout);
234         top.add(title, BorderLayout.WEST);
235         top.add(profileControls, BorderLayout.EAST);
236         return top;
237     }
238 
239     private void logout() {
240         if (approvalRefreshTimer != null) approvalRefreshTimer.stop();
241         new LoginFrame().setVisible(true);
242         dispose();
243     }
244 
245     private JPanel createRecordsPage() {
246         JPanel page = pagePanel(new BorderLayout(16, 16));
247         JPanel heading = heading("Waste records", "Log collection activity and follow each record's
        status.");
248         page.add(heading, BorderLayout.NORTH);
249 
250         JPanel content = new JPanel(new BorderLayout(14, 14));
251         content.setOpaque(false);
252         content.add(createRecordTablePanel(), BorderLayout.CENTER);
253         content.add(createRecordFormPanel(), BorderLayout.SOUTH);
254         page.add(content, BorderLayout.CENTER);
255         return page;
256     }
257 
258     private JScrollPane createRecordTablePanel() {
259         JPanel tableWrap = card(new BorderLayout(0, 12));
260         JPanel search = new JPanel(new BorderLayout(10, 0));
261         search.setOpaque(false);
262         searchField = input(new JTextField());
```

```
263         JButton find = button("Search", false);
264         JButton all = button("Refresh", false);
265         find.addActionListener(event -> refreshRecords());
266         all.addActionListener(event -> { searchField.setText(""); refreshRecords(); });
267         search.add(new JLabel("Search records"), BorderLayout.WEST);
268         search.add(searchField, BorderLayout.CENTER);
269         JPanel searchActions = new JPanel(new GridLayout(1, 2, 8, 0));
270         searchActions.setOpaque(false);
271         searchActions.add(find);
272         searchActions.add(all);
273         search.add(searchActions, BorderLayout.EAST);
274         tableWrap.add(search, BorderLayout.NORTH);
275
276         String[] columns = {"ID", "Type", "Description", "kg", "Collection area", "Collected at",
          "Collector", "Status"};
277         recordsModel = readOnlyModel(columns);
278         recordsTable = new JTable(recordsModel);
279         styleTable(recordsTable);
280         recordsTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
281         recordsTable.getSelectionModel().addListSelectionListener(event -> {
282             if (!event.getValueIsAdjusting() && recordsTable.getSelectedRow() >= 0)
          loadSelectedRecord();
283         });
284         JScrollPane scroll = new JScrollPane(recordsTable);
285         scroll.setBorder(BorderFactory.createLineBorder(LINE));
286         tableWrap.add(scroll, BorderLayout.CENTER);
287         return wrapCard(tableWrap);
288     }
289
290     private JScrollPane wrapCard(JPanel panel) {
291         JScrollPane wrapper = new JScrollPane(panel);
292         wrapper.setBorder(null);
293         wrapper.getViewport().setOpaque(false);
294         wrapper.setOpaque(false);
295         return wrapper;
296     }
297
298     private JPanel createRecordFormPanel() {
299         JPanel form = card(new BorderLayout(12, 12));
300         JLabel title = sectionTitle("Collection details");
301         form.add(title, BorderLayout.NORTH);
302
303         JPanel fields = new JPanel(new GridLayout(2, 4, 12, 8));
304         fields.setOpaque(false);
305         typeCombo = new JComboBox<>(new String[]{"Organic", "Recyclable", "Hazardous"});
306         descriptionField = input(new JTextField());
307         quantityField = input(new JTextField());
308         areaCombo = new JComboBox<>(AREAS);
309         areaCombo.setEditable(true);
310         collectedAtField = input(new JTextField(LocalDateTime.now().format(DATE_TIME_FORMAT)));
311         collectorCombo = new JComboBox<>();
312         loadCollectors();
313
314         fields.add(labeled("Waste type", typeCombo));
315         fields.add(labeled("Description", descriptionField));
316         fields.add(labeled("Weight (kg)", quantityField));
317         fields.add(labeled("Collection area", areaCombo));
318         fields.add(labeled("Collected at (yyyy-MM-dd HH:mm)", collectedAtField));
319         if (user.isAdmin()) fields.add(labeled("Attributed collector", collectorCombo));
320         else fields.add(labeled("Your assigned area", new JLabel(user.getAssignedArea())));
321         form.add(fields, BorderLayout.CENTER);
322
323         JPanel actions = new JPanel(new GridLayout(1, 0, 8, 0));
324         actions.setOpaque(false);
325         JButton add = button("Add record", true);
326         JButton update = button("Update selected", false);
327         JButton invalid = button("Mark invalid", false);
328         add.addActionListener(event -> addRecord());
329         update.addActionListener(event -> updateRecord());
330         invalid.addActionListener(event -> invalidateRecord());
331         actions.add(add);
332         actions.add(update);
333         actions.add(invalid);
334         if (user.isAdmin()) {
335             JButton complete = button("Mark as collected", true);
336             JButton delete = button("Delete selected", false);
337             complete.addActionListener(event -> completeRecord());
338             delete.addActionListener(event -> deleteRecord());
339             actions.add(complete);
340             actions.add(delete);
341         } else {
342             JButton request = button("Request approval", true);
343             request.addActionListener(event -> requestCompletion());
344             actions.add(request);
345         }
346         JButton clear = button("Clear", false);
347         clear.addActionListener(event -> clearForm());
348         actions.add(clear);
349         form.add(actions, BorderLayout.SOUTH);
350         return form;
```

```
351     }
352
353     private JPanel createReportsPage() {
354         JPanel page = pagePanel(new BorderLayout(16, 16));
355         page.add(createReportHeading(), BorderLayout.NORTH);
356         JPanel reportContent = new JPanel(new BorderLayout(14, 14));
357         reportContent.setOpaque(false);
358         reportContent.add(createSummaryCards(), BorderLayout.NORTH);
359         if (user.isAdmin()) reportContent.add(createAdminReports(), BorderLayout.CENTER);
360         else reportContent.add(createPersonalReport(), BorderLayout.CENTER);
361         page.add(reportContent, BorderLayout.CENTER);
362         return page;
363     }
364
365     private JPanel createApprovalsPage() {
366         JPanel page = pagePanel(new BorderLayout(16, 16));
367         JPanel heading = new JPanel(new BorderLayout(12, 0));
368         heading.setOpaque(false);
369         heading.add(heading("Collection approvals", "Member requests stay pending until an administrator
            approves them."), BorderLayout.WEST);
370         JButton refresh = button("Refresh requests", false);
371         refresh.addActionListener(event -> refreshApprovalRequests());
372         heading.add(refresh, BorderLayout.EAST);
373         page.add(heading, BorderLayout.NORTH);
374
375         JPanel panel = card(new BorderLayout(12, 12));
376         panel.add(sectionTitle("Waiting for review"), BorderLayout.NORTH);
377         completionRequestsModel = readOnlyModel(new String[]{"Request", "Record", "Type", "Description",
            "kg", "Actual area", "Collected at", "Requested by", "Requested at"});
378         completionRequestsTable = new JTable(completionRequestsModel);
379         styleTable(completionRequestsTable);
380         completionRequestsTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
381         panel.add(new JScrollPane(completionRequestsTable), BorderLayout.CENTER);
382         JPanel actions = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 0));
383         actions.setOpaque(false);
384         JButton reject = button("Reject request", false);
385         JButton approve = button("Approve and mark collected", true);
386         reject.addActionListener(event -> rejectCompletionRequest());
387         approve.addActionListener(event -> approveCompletionRequest());
388         actions.add(reject);
389         actions.add(approve);
390         panel.add(actions, BorderLayout.SOUTH);
391         page.add(panel, BorderLayout.CENTER);
392         return page;
393     }
394
395     private JPanel createMembersPage() {
396         JPanel page = pagePanel(new BorderLayout(16, 16));
397         page.add(heading("Member directory", "Registered Clean CET members and their assigned
            locations."), BorderLayout.NORTH);
398         JPanel panel = card(new BorderLayout(10, 10));
399         JPanel search = new JPanel(new BorderLayout(10, 0));
400         search.setOpaque(false);
401         memberSearchField = input(new JTextField());
402         JButton find = button("Search members", false);
403         JButton refresh = button("Refresh", false);
404         JButton addArea = button("Add registration area", true);
405         find.addActionListener(event -> refreshMembers());
406         refresh.addActionListener(event -> { memberSearchField.setText(""); refreshMembers(); });
407         addArea.addActionListener(event -> addRegistrationArea());
408         search.add(new JLabel("Search members"), BorderLayout.WEST);
409         search.add(memberSearchField, BorderLayout.CENTER);
410         JPanel actions = new JPanel(new GridLayout(1, 3, 8, 0));
411         actions.setOpaque(false);
412         actions.add(find);
413         actions.add(refresh);
414         actions.add(addArea);
415         search.add(actions, BorderLayout.EAST);
416         panel.add(search, BorderLayout.NORTH);
417         memberDirectoryModel = readOnlyModel(new String[]{"Name", "Username", "Date of birth", "CET
            register number",
418                 "Department", "Assigned area", "Account"});
419         JTable members = new JTable(memberDirectoryModel);
420         styleTable(members);
421         members.setAutoResizeMode(JTable.AUTO_RESIZE_OFF);
422         int[] widths = {150, 100, 115, 155, 170, 190, 85};
423         for (int i = 0; i < widths.length; i++)
            members.getColumnModel().getColumn(i).setPreferredWidth(widths[i]);
424         panel.add(new JScrollPane(members), BorderLayout.CENTER);
425         JLabel note = new JLabel("Passwords are stored as salted hashes and are never visible in the
            directory.");
426         note.setForeground(MUTED);
427         panel.add(note, BorderLayout.SOUTH);
428         page.add(panel, BorderLayout.CENTER);
429         return page;
430     }
431
432     private void refreshMembers() {
433         if (!user.isAdmin() || memberDirectoryModel == null) return;
434         try {
```

```
435             memberDirectoryModel.setRowCount(0);
436             for (Object[] member : userManager.getMembers(user, memberSearchField.getText())) {
437                 memberDirectoryModel.addRow(member);
438             }
439         } catch (SQLException exception) {
440             showError("Could not load member directory: " + exception.getMessage());
441         }
442     }
443 
444     private void addRegistrationArea() {
445         String area = JOptionPane.showInputDialog(this, "New area for future member assignment:",
446                 "Add registration area", JOptionPane.QUESTION_MESSAGE);
447         if (area == null) return;
448         try {
449             userManager.addRegistrationArea(area, user);
450             showMessage("Registration area added. It is now available for random assignment to a new
      member.");
451         } catch (IllegalArgumentException | SQLException exception) {
452             showError(exception.getMessage());
453         }
454     }
455 
456     private JPanel createAnnouncementsPage() {
457         JPanel page = pagePanel(new BorderLayout(16, 16));
458         JPanel heading = new JPanel(new BorderLayout(12, 0));
459         heading.setOpaque(false);
460         heading.add(heading("Announcements", "Updates from the Clean CET administrator."),
      BorderLayout.WEST);
461         if (user.isAdmin()) {
462             JButton publish = button("New announcement", true);
463             publish.addActionListener(event -> publishAnnouncement());
464             heading.add(publish, BorderLayout.EAST);
465         }
466         page.add(heading, BorderLayout.NORTH);
467 
468         JPanel content = new JPanel(new BorderLayout(14, 0));
469         content.setOpaque(false);
470         announcementsModel = readOnlyModel(new String[]{"Title", "From", "Date", "Status"});
471         announcementsTable = new JTable(announcementsModel);
472         styleTable(announcementsTable);
473         announcementsTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
474         announcementsTable.getSelectionModel().addListSelectionListener(event -> {
475             if (!event.getValueIsAdjusting() && announcementsTable.getSelectedRow() >= 0) {
476 
      showAnnouncement(announcementsTable.convertRowIndexToModel(announcementsTable.getSelectedRow()));
477             }
478         });
479         JPanel list = card(new BorderLayout(0, 10));
480         list.add(sectionTitle("Message inbox"), BorderLayout.NORTH);
481         list.add(new JScrollPane(announcementsTable), BorderLayout.CENTER);
482         JPanel details = card(new BorderLayout(0, 10));
483         details.add(sectionTitle("Announcement details"), BorderLayout.NORTH);
484         announcementDetails = new JTextArea();
485         announcementDetails.setEditable(false);
486         announcementDetails.setLineWrap(true);
487         announcementDetails.setWrapStyleWord(true);
488         announcementDetails.setFont(new Font("SansSerif", Font.PLAIN, 13));
489         announcementDetails.setForeground(INK);
490         announcementDetails.setBorder(new EmptyBorder(10, 10, 10, 10));
491         details.add(new JScrollPane(announcementDetails), BorderLayout.CENTER);
492         content.add(list, BorderLayout.CENTER);
493         content.add(details, BorderLayout.EAST);
494         details.setPreferredSize(new Dimension(330, 0));
495         page.add(content, BorderLayout.CENTER);
496         return page;
497     }
498 
499     private void refreshAnnouncements() {
500         if (announcementsModel == null) return;
501         try {
502             announcements = announcementManager.getAnnouncements(user);
503             announcementsModel.setRowCount(0);
504             for (Announcement announcement : announcements) {
505                 announcementsModel.addRow(new Object[]{announcement.getTitle(),
      announcement.getAuthor(),
506                         announcement.getCreatedAt().format(DateTimeFormatter.ofPattern("yyyy-MM-dd
      HH:mm")),
507                         user.isAdmin() ? "Published" : announcement.isRead() ? "Read" : "New"});
508             }
509             announcementsNavigation.setText("Announcements (" + announcementManager.getUnreadCount(user)
       + ")");
510             if (announcements.isEmpty()) announcementDetails.setText("No announcements yet.");
511         } catch (SQLException exception) {
512             showError("Could not load announcements: " + exception.getMessage());
513         }
514     }
515 
516     private void showAnnouncement(int index) {
517         if (index < 0 || index >= announcements.size()) return;
518         Announcement announcement = announcements.get(index);
```

```
519         announcementDetails.setText(announcement.getTitle() + "\n\n"
520                 + announcement.getMessage() + "\n\nFrom: " + announcement.getAuthor()
521                 + "\nPosted: " + announcement.getCreatedAt().format(DateTimeFormatter.ofPattern("yyyy-
          MM-dd HH:mm")));
522         if (!user.isAdmin() && !announcement.isRead()) {
523             try {
524                 announcementManager.markRead(announcement.getId(), user);
525                 refreshAnnouncements();
526                 announcementsTable.setRowSelectionInterval(index, index);
527             } catch (SQLException exception) {
528                 showError("Could not mark announcement as read: " + exception.getMessage());
529             }
530         }
531     }
532
533     private void publishAnnouncement() {
534         JTextField title = input(new JTextField());
535         JTextArea message = new JTextArea(6, 28);
536         message.setLineWrap(true);
537         message.setWrapStyleWord(true);
538         JPanel form = new JPanel(new BorderLayout(0, 8));
539         form.add(labeled("Title", title), BorderLayout.NORTH);
540         form.add(labeled("Message", new JScrollPane(message)), BorderLayout.CENTER);
541         if (JOptionPane.showConfirmDialog(this, form, "New announcement",
542                 JOptionPane.OK_CANCEL_OPTION, JOptionPane.PLAIN_MESSAGE) != JOptionPane.OK_OPTION)
          return;
543         try {
544             announcementManager.publish(title.getText(), message.getText(), user);
545             refreshAnnouncements();
546             showMessage("Announcement sent to all users.");
547         } catch (SQLException exception) {
548             showError(exception.getMessage());
549         }
550     }
551
552     private JPanel createReportHeading() {
553         JPanel area = new JPanel(new BorderLayout(12, 10));
554         area.setOpaque(false);
555         JPanel title = heading(user.isAdmin() ? "Reports dashboard" : "My collection progress",
556                 user.isAdmin() ? "Completed collection totals by member and area." : "Your completed
          collection and additional contribution.");
557         area.add(title, BorderLayout.WEST);
558         JPanel filters = new JPanel(new GridLayout(2, 2, 8, 4));
559         filters.setOpaque(false);
560         fromField = input(new JTextField(LocalDate.now().withDayOfMonth(1).toString()));
561         toField = input(new JTextField(LocalDate.now().toString()));
562         JButton apply = button("Apply dates", true);
563         JButton export = button("Export CSV", false);
564         apply.addActionListener(event -> applyReportDates());
565         export.addActionListener(event -> exportCurrentReport());
566         filters.add(new JLabel("From date")); filters.add(new JLabel("To date"));
567         filters.add(fromField); filters.add(toField);
568         JPanel control = new JPanel(new BorderLayout(8, 0));
569         control.setOpaque(false);
570         control.add(filters, BorderLayout.CENTER);
571         apply.setPreferredSize(new Dimension(108, 40));
572         export.setPreferredSize(new Dimension(108, 40));
573         JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT, 8, 2));
574         buttons.setOpaque(false); buttons.add(apply); buttons.add(export);
575         control.add(buttons, BorderLayout.EAST);
576         appliedRangeLabel = new JLabel("Applied range: not applied");
577         appliedRangeLabel.setForeground(MUTED);
578         control.add(appliedRangeLabel, BorderLayout.SOUTH);
579         area.add(control, BorderLayout.EAST);
580         return area;
581     }
582
583     private JPanel createSummaryCards() {
584         JPanel cards = new JPanel(new GridLayout(1, 5, 12, 0));
585         cards.setOpaque(false);
586         totalValue = new JLabel("0 kg");
587         organicValue = new JLabel("0 kg");
588         recyclableValue = new JLabel("0 kg");
589         hazardousValue = new JLabel("0 kg");
590         recordCountValue = new JLabel("0");
591         cards.add(metric("Total collected", totalValue));
592         cards.add(metric("Organic", organicValue));
593         cards.add(metric("Recyclable", recyclableValue));
594         cards.add(metric("Hazardous", hazardousValue));
595         cards.add(metric("Completed records", recordCountValue));
596         return cards;
597     }
598
599     private JPanel createAdminReports() {
600         JTabbedPane tabs = new JTabbedPane();
601         tabs.addTab("Member reports", createMemberReportPanel());
602         tabs.addTab("Area reports", createAreaReportPanel());
603         tabs.setFont(new Font("SansSerif", Font.BOLD, 12));
604         JPanel wrapper = new JPanel(new BorderLayout());
605         wrapper.setOpaque(false);
```

```
606         wrapper.add(tabs, BorderLayout.CENTER);
607         return wrapper;
608     }
609
610     private JPanel createMemberReportPanel() {
611         JPanel panel = card(new BorderLayout(12, 12));
612         panel.add(sectionTitle("Individual contribution by member"), BorderLayout.NORTH);
613         String[] cols = {"Member", "Total kg", "Organic kg", "Recyclable kg", "Hazardous kg", "Assigned
      area kg", "Bonus kg", "Records"};
614         memberReportModel = readOnlyModel(cols);
615         memberReportTable = new JTable(memberReportModel);
616         styleTable(memberReportTable);
617         panel.add(new JScrollPane(memberReportTable), BorderLayout.CENTER);
618         return panel;
619     }
620
621     private JPanel createAreaReportPanel() {
622         JPanel panel = new JPanel(new BorderLayout(12, 12));
623         panel.setOpaque(false);
624         JPanel chartPanel = card(new BorderLayout(8, 8));
625         chartPanel.add(sectionTitle("Collected weight by area (kg)"), BorderLayout.NORTH);
626         areaChart = new AreaChart();
627         chartPanel.add(areaChart, BorderLayout.CENTER);
628         chartPanel.setPreferredSize(new Dimension(310, 500));
629
630         JPanel tableCard = card(new BorderLayout(10, 10));
631         tableCard.add(sectionTitle("Collected weight and record count by area"), BorderLayout.NORTH);
632         areaReportModel = readOnlyModel(new String[]{"Area", "Total kg", "Organic kg", "Recyclable kg",
      "Hazardous kg", "Records"});
633         areaReportTable = new JTable(areaReportModel);
634         styleTable(areaReportTable);
635         tableCard.add(new JScrollPane(areaReportTable), BorderLayout.CENTER);
636         panel.add(chartPanel, BorderLayout.WEST);
637         panel.add(tableCard, BorderLayout.CENTER);
638         return panel;
639     }
640
641     private JPanel createPersonalReport() {
642         JPanel panel = card(new BorderLayout(12, 12));
643         panel.add(sectionTitle("Your individual report"), BorderLayout.NORTH);
644         personalReportModel = readOnlyModel(new String[]{"Contribution", "Total kg", "Organic kg",
      "Recyclable kg", "Hazardous kg", "Records"});
645         personalReportTable = new JTable(personalReportModel);
646         styleTable(personalReportTable);
647         panel.add(new JScrollPane(personalReportTable), BorderLayout.CENTER);
648         JLabel note = new JLabel("Assigned area and bonus collection are shown separately in the
      contribution rows.");
649         note.setForeground(MUTED);
650         panel.add(note, BorderLayout.SOUTH);
651         return panel;
652     }
653
654     private void loadCollectors() {
655         if (!user.isAdmin() || collectorCombo == null) return;
656         try {
657             collectorCombo.removeAllItems();
658             collectorCombo.addItem("Unassigned (admin entry)");
659             for (String[] collector : wasteManager.getCollectors()) collectorCombo.addItem(collector[1]
      + " (" + collector[0] + ")");
660         } catch (SQLException exception) {
661             showError(exception.getMessage());
662         }
663     }
664
665     private String selectedCollectorUsername() {
666         if (!user.isAdmin() || collectorCombo.getSelectedIndex() <= 0) return null;
667         String selected = (String) collectorCombo.getSelectedItem();
668         int start = selected.lastIndexOf('(');
669         return selected.substring(start + 1, selected.length() - 1);
670     }
671
672     private void addRecord() {
673         try {
674             Waste waste = wasteFromForm(0);
675             wasteManager.addWaste(waste, user, selectedCollectorUsername());
676             showMessage(user.isAdmin()
677                     ? "Waste record added as pending. Mark it collected or approve a member request to
      include it in reports."
678                     : "Waste record added as pending. Request collection approval after reviewing the
      record.");
679             clearForm();
680             refreshAll();
681         } catch (IllegalArgumentException exception) {
682             showError(exception.getMessage());
683         } catch (SQLException exception) {
684             showError(exception.getMessage());
685         }
686     }
687
688     private void updateRecord() {
```

```
689         if (selectedId < 0) { showError("Select one of your pending records first."); return; }
690         try {
691             wasteManager.updateWaste(wasteFromForm(selectedId), user, selectedCollectorUsername());
692             showMessage("Pending record updated.");
693             clearForm();
694             refreshAll();
695         } catch (IllegalArgumentException | SQLException exception) {
696             showError(exception.getMessage());
697         }
698     }
699
700     private void invalidateRecord() {
701         if (selectedId < 0) { showError("Select a pending record first."); return; }
702         String reason = JOptionPane.showInputDialog(this, "Why is this record invalid?", "Mark invalid",
            JOptionPane.QUESTION_MESSAGE);
703         if (reason == null) return;
704         try {
705             wasteManager.markInvalid(selectedId, reason, user);
706             clearForm();
707             refreshAll();
708         } catch (SQLException exception) { showError(exception.getMessage()); }
709     }
710
711     private void requestCompletion() {
712         if (selectedId < 0) { showError("Select one of your pending records first."); return; }
713         try {
714             wasteManager.requestCompletion(selectedId, user);
715             clearForm();
716             refreshAll();
717             showMessage("Completion request sent. The record will count in reports after admin
          approval.");
718         } catch (SQLException exception) { showError(exception.getMessage()); }
719     }
720
721     private void completeRecord() {
722         if (!user.isAdmin()) return;
723         if (selectedId < 0) { showError("Select a pending record first."); return; }
724         try {
725             wasteManager.markCompleted(selectedId, user);
726             clearForm();
727             refreshAll();
728             showMessage("Record marked as collected. Reports have been updated.");
729         } catch (SQLException exception) { showError(exception.getMessage()); }
730     }
731
732     private void refreshApprovalCount() {
733         if (!user.isAdmin() || completionRequestsNavigation == null) return;
734         try {
735             completionRequestsNavigation.setText("Collection approvals (" +
          wasteManager.getPendingCompletionRequestCount(user) + ")");
736         } catch (SQLException exception) {
737             completionRequestsNavigation.setToolTipText("Could not refresh approval count: " +
          exception.getMessage());
738         }
739     }
740
741     private void refreshApprovalRequests() {
742         if (!user.isAdmin() || completionRequestsModel == null) return;
743         try {
744             completionRequestsModel.setRowCount(0);
745             for (Object[] request : wasteManager.getPendingCompletionRequests(user))
          completionRequestsModel.addRow(request);
746             refreshApprovalCount();
747         } catch (SQLException exception) { showError("Could not load collection approvals: " +
          exception.getMessage()); }
748     }
749
750     private Long selectedRequestId() {
751         int row = completionRequestsTable.getSelectedRow();
752         if (row < 0) {
753             showError("Select a completion request first.");
754             return null;
755         }
756         return ((Number)
          completionRequestsModel.getValueAt(completionRequestsTable.convertRowIndexToModel(row),
          0)).longValue();
757     }
758
759     private void approveCompletionRequest() {
760         Long requestId = selectedRequestId();
761         if (requestId == null) return;
762         try {
763             wasteManager.approveCompletionRequest(requestId, user);
764             refreshAll();
765             showMessage("Request approved. The record is now marked collected and included in
          reports.");
766         } catch (SQLException exception) { showError(exception.getMessage()); }
767     }
768
769     private void rejectCompletionRequest() {
```

```
770         Long requestId = selectedRequestId();
771         if (requestId == null) return;
772         try {
773             wasteManager.rejectCompletionRequest(requestId, user);
774             refreshApprovalRequests();
775             showMessage("Request rejected. The waste record remains pending and can be corrected or
          resubmitted.");
776         } catch (SQLException exception) { showError(exception.getMessage()); }
777     }
778
779     private void deleteRecord() {
780         if (!user.isAdmin()) return;
781         if (selectedId < 0) { showError("Select a record first."); return; }
782         int answer = JOptionPane.showConfirmDialog(this, "Permanently delete record #" + selectedId +
          "?",
783                 "Confirm delete", JOptionPane.YES_NO_OPTION, JOptionPane.WARNING_MESSAGE);
784         if (answer != JOptionPane.YES_OPTION) return;
785         try {
786             wasteManager.deleteWaste(selectedId, user);
787             clearForm();
788             refreshAll();
789         } catch (SQLException exception) { showError(exception.getMessage()); }
790     }
791
792     private Waste wasteFromForm(int id) {
793         String description = descriptionField.getText().trim();
794         if (description.isEmpty()) throw new IllegalArgumentException("Description is required.");
795         double quantity;
796         try { quantity = Double.parseDouble(quantityField.getText().trim()); }
797         catch (NumberFormatException exception) { throw new IllegalArgumentException("Enter a valid
          weight in kilograms."); }
798         if (!Double.isFinite(quantity) || quantity <= 0) throw new IllegalArgumentException("Weight must
           be greater than zero.");
799         String area = String.valueOf(areaCombo.getEditor().getItem()).trim();
800         if (area.isEmpty()) throw new IllegalArgumentException("Collection area is required.");
801         LocalDateTime collectedAt;
802         try { collectedAt = LocalDateTime.parse(collectedAtField.getText().trim(), DATE_TIME_FORMAT); }
803         catch (DateTimeParseException exception) { throw new IllegalArgumentException("Use timestamp
          format yyyy-MM-dd HH:mm."); }
804         if (collectedAt.isAfter(LocalDateTime.now().plusMinutes(1))) throw new
          IllegalArgumentException("Collection timestamp cannot be in the future.");
805         String type = (String) typeCombo.getSelectedItem();
806         String status = selectedStatus == null ? "PENDING" : selectedStatus;
807         Waste waste;
808         if ("Organic".equals(type)) waste = new model.OrganicWaste(id, description, quantity, area,
          status);
809         else if ("Recyclable".equals(type)) waste = new model.RecyclableWaste(id, description, quantity,
           area, status);
810         else waste = new model.HazardousWaste(id, description, quantity, area, status);
811         waste.setCollectedAt(collectedAt.toString());
812         return waste;
813     }
814
815     private void loadSelectedRecord() {
816         int row = recordsTable.convertRowIndexToModel(recordsTable.getSelectedRow());
817         selectedId = (Integer) recordsModel.getValueAt(row, 0);
818         typeCombo.setSelectedItem(recordsModel.getValueAt(row, 1));
819         descriptionField.setText(String.valueOf(recordsModel.getValueAt(row, 2)));
820         quantityField.setText(String.valueOf(recordsModel.getValueAt(row, 3)));
821         areaCombo.setSelectedItem(recordsModel.getValueAt(row, 4));
822         collectedAtField.setText(String.valueOf(recordsModel.getValueAt(row, 5)));
823         selectedStatus = String.valueOf(recordsModel.getValueAt(row, 7));
824         if (user.isAdmin()) {
825             collectorCombo.setSelectedIndex(0);
826         }
827         if (user.isAdmin() && recordsModel.getValueAt(row, 6) != null) {
828             String collector = String.valueOf(recordsModel.getValueAt(row, 6));
829             for (int i = 0; i < collectorCombo.getItemCount(); i++) {
830                 if (collectorCombo.getItemAt(i).contains("(" + collector + ")"))
          collectorCombo.setSelectedIndex(i);
831             }
832         }
833     }
834
835     private void clearForm() {
836         selectedId = -1;
837         selectedStatus = null;
838         descriptionField.setText("");
839         quantityField.setText("");
840         typeCombo.setSelectedIndex(0);
841         areaCombo.setSelectedItem(user.isAdmin() ? AREAS[0] : user.getAssignedArea());
842         collectedAtField.setText(LocalDateTime.now().format(DATE_TIME_FORMAT));
843         if (user.isAdmin() && collectorCombo.getItemCount() > 0) collectorCombo.setSelectedIndex(0);
844         recordsTable.clearSelection();
845     }
846
847     private void refreshAll() {
848         refreshRecords();
849         refreshReports();
850         refreshApprovalRequests();
```

```
851     }
852
853     private void refreshRecords() {
854         if (recordsModel == null) return;
855         try {
856             recordsModel.setRowCount(0);
857             for (Waste waste : wasteManager.getVisibleWaste(user, searchField == null ? "" :
      searchField.getText())) {
858                 recordsModel.addRow(new Object[]{waste.getId(), waste.getWasteCategory(),
      waste.getDescription(),
859                         waste.getQuantity(), waste.getLocation(), waste.getCollectedAt(),
      waste.getCollectedBy(),
860                         displayStatus(waste)});
861             }
862         } catch (SQLException exception) { showError(exception.getMessage()); }
863     }
864
865     private void refreshReports() {
866         applyReportDates();
867     }
868
869     private void applyReportDates() {
870         if (totalValue == null) return;
871         try {
872             LocalDate from = LocalDate.parse(fromField.getText().trim());
873             LocalDate to = LocalDate.parse(toField.getText().trim());
874             if (from.isAfter(to)) throw new IllegalArgumentException("From date must be on or before To
      date.");
875             ReportSummary summary = reportManager.getSummary(user, from, to);
876             List<ReportRow> memberRows = null;
877             List<ReportRow> areaRows = null;
878             ReportRow own = null;
879             if (user.isAdmin()) {
880                 memberRows = reportManager.getAllUserReports(from, to, user);
881                 areaRows = reportManager.getAreaReports(from, to, user);
882             } else {
883                 own = reportManager.getUserReport(user.getUsername(), from, to, user);
884             }
885
886             totalValue.setText(kg(summary.getTotalKg()));
887             organicValue.setText(kg(summary.getOrganicKg()));
888             recyclableValue.setText(kg(summary.getRecyclableKg()));
889             hazardousValue.setText(kg(summary.getHazardousKg()));
890             recordCountValue.setText(String.valueOf(summary.getCompletedRecords()));
891             if (user.isAdmin()) {
892                 currentMemberReports = memberRows;
893                 currentAreaReports = areaRows;
894                 fillMemberRows();
895                 fillAreaRows();
896                 areaChart.setReports(currentAreaReports);
897             } else {
898                 personalReportModel.setRowCount(0);
899                 personalReportModel.addRow(new Object[]{"Assigned area", kg(own.getAssignedKg()), "-",
      "-", "-", "-"});
900                 personalReportModel.addRow(new Object[]{"Additional / bonus", kg(own.getBonusKg()), "-",
       "-", "-", "-"});
901                 personalReportModel.addRow(new Object[]{"All your collections", kg(own.getTotalKg()),
      kg(own.getOrganic()),
902                         kg(own.getRecyclable()), kg(own.getHazardous()), own.getRecords()});
903             }
904             appliedFrom = from;
905             appliedTo = to;
906             appliedRangeLabel.setText("Applied range: " + from + " to " + to);
907         } catch (DateTimeParseException exception) {
908             showError("Enter report dates using yyyy-MM-dd.");
909         } catch (IllegalArgumentException | SQLException exception) {
910             showError(exception.getMessage());
911         }
912     }
913
914     private void fillMemberRows() {
915         memberReportModel.setRowCount(0);
916         for (ReportRow row : currentMemberReports) memberReportModel.addRow(new Object[]{row.getLabel(),
       kg(row.getTotalKg()),
917                 kg(row.getOrganic()), kg(row.getRecyclable()), kg(row.getHazardous()),
      kg(row.getAssignedKg()),
918                 kg(row.getBonusKg()), row.getRecords()});
919     }
920
921     private static String displayStatus(Waste waste) {
922         if (waste.isCompletionRequested()) return "PENDING - AWAITING APPROVAL";
923         if ("REJECTED".equals(waste.getCompletionRequestStatus())) return "PENDING - REQUEST REJECTED";
924         return waste.getStatus();
925     }
926
927     private void fillAreaRows() {
928         areaReportModel.setRowCount(0);
929         for (ReportRow row : currentAreaReports) areaReportModel.addRow(new Object[]{row.getLabel(),
      kg(row.getTotalKg()),
930                 kg(row.getOrganic()), kg(row.getRecyclable()), kg(row.getHazardous()),
```

```
          row.getRecords()});
931       }
932
933       private void exportCurrentReport() {
934           if (appliedFrom == null || appliedTo == null) {
935               showError("Apply a valid date range before exporting.");
936               return;
937           }
938           JFileChooser chooser = new JFileChooser();
939           chooser.setSelectedFile(new File(user.isAdmin() ? "clean-cet-waste-report.csv" :
          user.getUsername() + "-waste-report.csv"));
940           chooser.setFileFilter(new FileNameExtensionFilter("CSV files (*.csv)", "csv"));
941           if (chooser.showSaveDialog(this) != JFileChooser.APPROVE_OPTION) return;
942           try {
943               LocalDate from = appliedFrom;
944               LocalDate to = appliedTo;
945               List<ReportRow> memberRows = user.isAdmin() ? reportManager.getAllUserReports(from, to,
          user) : null;
946               List<ReportRow> areaRows = user.isAdmin() ? reportManager.getAreaReports(from, to, user) :
          null;
947               ReportRow personalRow = user.isAdmin() ? null
948                       : reportManager.getUserReport(user.getUsername(), from, to, user);
949               File output = chooser.getSelectedFile();
950               if (!output.getName().toLowerCase(java.util.Locale.ROOT).endsWith(".csv")) {
951                   output = new File(output.getAbsolutePath() + ".csv");
952               }
953               if (output.exists() && JOptionPane.showConfirmDialog(this,
954                       "Replace " + output.getName() + "?", "Confirm export", JOptionPane.YES_NO_OPTION,
955                       JOptionPane.WARNING_MESSAGE) != JOptionPane.YES_OPTION) return;
956               try (BufferedWriter writer = Files.newBufferedWriter(output.toPath(),
          StandardCharsets.UTF_8)) {
957                   writer.write("Report period," + csv(from.toString()) + "," + csv(to.toString()) +
          "\n\n");
958                   if (user.isAdmin()) {
959                       writer.write("Member,Total kg,Organic kg,Recyclable kg,Hazardous kg,Assigned area
          kg,Bonus kg,Completed records\n");
960                       for (ReportRow row : memberRows) writer.write(csv(row.getLabel()) + "," +
          row.getTotalKg() + "," + row.getOrganic() + "," + row.getRecyclable() + "," + row.getHazardous() +
           "," + row.getAssignedKg() + "," + row.getBonusKg() + "," + row.getRecords() + "\n");
961                       writer.write("\nArea,Total kg,Organic kg,Recyclable kg,Hazardous kg,Completed
          records\n");
962                       for (ReportRow row : areaRows) writer.write(csv(row.getLabel()) + "," +
          row.getTotalKg() + "," + row.getOrganic() + "," + row.getRecyclable() + "," + row.getHazardous() +
           "," + row.getRecords() + "\n");
963                   } else {
964                       writer.write("Member,Total kg,Organic kg,Recyclable kg,Hazardous kg,Assigned area
          kg,Bonus kg,Completed records\n");
965                       writer.write(csv(user.getDisplayName()) + "," + personalRow.getTotalKg() + "," +
          personalRow.getOrganic() + "," + personalRow.getRecyclable() + "," + personalRow.getHazardous() +
          "," + personalRow.getAssignedKg() + "," + personalRow.getBonusKg() + "," +
          personalRow.getRecords() + "\n");
966                   }
967               }
968               showMessage("Report exported successfully.");
969           } catch (Exception exception) { showError("Could not export report: " + exception.getMessage());
           }
970       }
971
972       private static String csv(String value) {
973           String safe = value == null ? "" : value;
974           if (safe.matches("^[\\s]*[=+@\\-].*")) safe = "'" + safe;
975           return "\"" + safe.replace("\"", "\"\"") + "\"";
976       }
977       private static String kg(double amount) { return String.format(java.util.Locale.ROOT, "%.2f kg",
          amount); }
978
979       private JPanel pagePanel(java.awt.LayoutManager layout) {
980           JPanel panel = new JPanel(layout);
981           panel.setBackground(PAGE);
982           panel.setBorder(new EmptyBorder(22, 24, 24, 24));
983           return panel;
984       }
985
986       private JPanel heading(String title, String subtitle) {
987           JPanel panel = new JPanel(new GridLayout(2, 1, 0, 4));
988           panel.setOpaque(false);
989           JLabel titleLabel = sectionTitle(title);
990           JLabel subtitleLabel = new JLabel(subtitle);
991           subtitleLabel.setForeground(MUTED);
992           panel.add(titleLabel);
993           panel.add(subtitleLabel);
994           return panel;
995       }
996
997       private JLabel sectionTitle(String title) {
998           JLabel label = new JLabel(title);
999           label.setFont(new Font("SansSerif", Font.BOLD, 17));
1000          label.setForeground(INK);
1001          return label;
1002      }
```

```
1003
1004      private JPanel card(java.awt.LayoutManager layout) {
1005          JPanel panel = new JPanel(layout);
1006          panel.setBackground(Color.WHITE);
1007          panel.setBorder(BorderFactory.createCompoundBorder(BorderFactory.createLineBorder(LINE),
1008                  new EmptyBorder(15, 16, 15, 16)));
1009          return panel;
1010      }
1011
1012      private JPanel metric(String name, JLabel value) {
1013          JPanel panel = card(new GridLayout(2, 1, 0, 8));
1014          JLabel caption = new JLabel(name);
1015          caption.setForeground(MUTED);
1016          value.setForeground(PURPLE_DARK);
1017          value.setFont(new Font("SansSerif", Font.BOLD, 21));
1018          panel.add(caption);
1019          panel.add(value);
1020          return panel;
1021      }
1022
1023      private JPanel labeled(String name, java.awt.Component component) {
1024          JPanel panel = new JPanel(new BorderLayout(0, 5));
1025          panel.setOpaque(false);
1026          JLabel label = new JLabel(name);
1027          label.setForeground(MUTED);
1028          label.setFont(new Font("SansSerif", Font.PLAIN, 12));
1029          panel.add(label, BorderLayout.NORTH);
1030          panel.add(component, BorderLayout.CENTER);
1031          return panel;
1032      }
1033
1034      private JTextField input(JTextField field) {
1035          field.setFont(new Font("SansSerif", Font.PLAIN, 13));
1036          field.setBorder(BorderFactory.createCompoundBorder(BorderFactory.createLineBorder(LINE),
1037                  new EmptyBorder(8, 9, 8, 9)));
1038          return field;
1039      }
1040
1041      private JButton button(String text, boolean primary) {
1042          JButton button = new JButton(text);
1043          button.setUI(new CleanButtonUI());
1044          button.setOpaque(true);
1045          button.setContentAreaFilled(true);
1046          button.setBorderPainted(true);
1047          button.setBackground(primary ? PURPLE : new Color(248, 247, 251));
1048          button.setForeground(primary ? Color.WHITE : PURPLE_DARK);
1049          button.setFont(new Font("SansSerif", Font.BOLD, 11));
1050          button.setFocusPainted(false);
1051          button.setMargin(new Insets(5, 6, 5, 6));
1052          button.setMinimumSize(new Dimension(0, 38));
1053          button.setBorder(BorderFactory.createCompoundBorder(BorderFactory.createLineBorder(primary ?
        PURPLE : LINE),
1054                  new EmptyBorder(6, 8, 6, 8)));
1055          return button;
1056      }
1057
1058      private static DefaultTableModel readOnlyModel(String[] columns) {
1059          return new DefaultTableModel(columns, 0) {
1060              @Override public boolean isCellEditable(int row, int column) { return false; }
1061          };
1062      }
1063
1064      private static void styleTable(JTable table) {
1065          table.setRowHeight(31);
1066          table.setFont(new Font("SansSerif", Font.PLAIN, 12));
1067          table.setForeground(INK);
1068          table.setSelectionBackground(PURPLE_LIGHT);
1069          table.setSelectionForeground(PURPLE_DARK);
1070          table.setShowGrid(false);
1071          table.setIntercellSpacing(new Dimension(0, 0));
1072          table.getTableHeader().setFont(new Font("SansSerif", Font.BOLD, 12));
1073          table.getTableHeader().setBackground(PURPLE);
1074          table.getTableHeader().setForeground(Color.WHITE);
1075          table.getTableHeader().setPreferredSize(new Dimension(0, 36));
1076          table.getTableHeader().setReorderingAllowed(false);
1077      }
1078
1079      private void showMessage(String message) { JOptionPane.showMessageDialog(this, message); }
1080      private void showError(String message) { JOptionPane.showMessageDialog(this, message, "Waste
        Management", JOptionPane.ERROR_MESSAGE); }
1081
1082      private static class AreaChart extends JPanel {
1083          private List<ReportRow> reports = new ArrayList<>();
1084
1085          AreaChart() { setBackground(Color.WHITE); setPreferredSize(new Dimension(260, 440)); }
1086
1087          void setReports(List<ReportRow> reports) { this.reports = new ArrayList<>(reports); repaint(); }
1088
1089          @Override protected void paintComponent(java.awt.Graphics graphics) {
1090              super.paintComponent(graphics);
```

```
1091              java.awt.Graphics2D g = (java.awt.Graphics2D) graphics.create();
1092              g.setRenderingHint(java.awt.RenderingHints.KEY_ANTIALIASING,
          java.awt.RenderingHints.VALUE_ANTIALIAS_ON);
1093              if (reports.isEmpty()) {
1094                  g.setColor(MUTED);
1095                  g.setFont(new Font("SansSerif", Font.PLAIN, 12));
1096                  g.drawString("No completed collections in this date range.", 12, 24);
1097                  g.dispose();
1098                  return;
1099              }
1100              int left = 10, right = 42, top = 12;
1101              int rowHeight = Math.max(24, Math.min(29, (getHeight() - top) / reports.size()));
1102              double max = 1;
1103              for (ReportRow row : reports) max = Math.max(max, row.getTotalKg());
1104              g.setFont(new Font("SansSerif", Font.PLAIN, 10));
1105              for (int i = 0; i < reports.size(); i++) {
1106                  ReportRow row = reports.get(i);
1107                  int y = top + i * rowHeight;
1108                  String label = row.getLabel().length() > 17 ? row.getLabel().substring(0, 16) + "…" :
          row.getLabel();
1109                  g.setColor(INK);
1110                  g.drawString(label, left, y + 10);
1111                  int barX = left;
1112                  int barY = y + 15;
1113                  int width = Math.max(2, (int) ((getWidth() - left - right) * row.getTotalKg() / max));
1114                  g.setColor(PURPLE_LIGHT);
1115                  g.fillRoundRect(barX, barY, Math.max(2, getWidth() - left - right), 8, 8, 8);
1116                  g.setColor(PURPLE);
1117                  g.fillRoundRect(barX, barY, width, 8, 8, 8);
1118                  g.setColor(MUTED);
1119                  g.drawString(String.format(java.util.Locale.ROOT, "%.1f", row.getTotalKg()), getWidth()
          - right + 5, barY + 8);
1120              }
1121              g.dispose();
1122          }
1123      }
1124  }
```